กำหนดการพลวัติสำหรับการโปรแกรมเชิงแข่งขัน (ฉบับร่าง-01)

Dynamic Programming for Competitive Programming

จักริน ชวชาติ
ภาควิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยเชียงใหม่

March 11, 2026

บทที่ 1

บทนำ

ในปัจจุบันการแก้ปัญหาด้วยคอมพิวเตอร์หรือ “การโปรแกรมเชิงแข่งขัน” (Competitive Programming) ได้รับความนิยมอย่างแพร่หลาย ไม่ว่าจะเป็นในเวทีการแข่งขันทั้งระดับประเทศหรือระดับนานาชาติ หรือแม้แต่การสอบสัมภาษณ์เข้าทำงานในบริษัทเทคโนโลยีชั้นนำ

โปรแกรมเมอร์หรือนักแก้ปัญหาเชิงแข่งขัน จำเป็นต้องมีทักษะที่สำคัญ คือการวิเคราะห์ปัญหา เลือกวิธีแก้ที่มีประสิทธิภาพ และสามารถออกแบบอัลกอริทึมที่เหมาะสมกับบริบทของโจทย์ ในการแข่งขัน มักต้องเผชิญกับปัญหาต่าง ๆ เช่น การหาค่าที่เหมาะสมที่สุด (Optimization), การค้นหาและเรียงลำดับข้อมูล (Searching & Sorting), การจัดการกับข้อมูลในรูปแบบโครงสร้างข้อมูล (Data Structures), การหาความเป็นไปได้ต่าง ๆ (Combinatorics) ฯลฯ โดยที่การแก้ปัญหาที่ดีไม่ได้หมายถึง “แค่ให้ได้คำตอบที่ถูกต้อง” เท่านั้น แต่ต้อง “ให้ได้คำตอบที่ถูกต้องในเวลาที่กำหนด” อีกด้วย เพราะโปรแกรมที่ทำงานช้าหรือใช้เวลามากเกินไปในชุดข้อมูลขนาดใหญ่ จะไม่ผ่านเกณฑ์ของการแข่งขัน ซึ่งมักกำหนดด้วยขอบเขตของขนาดข้อมูล (Input Size) เอาไว้

ในการแก้ปัญหาเชิงอัลกอริทึมในเบื้องต้น หลายคนอาจเริ่มจากแนวทางง่าย ๆ อย่าง Brute Force คือ ทดลองทุกความเป็นไปได้แล้วเลือกคำตอบที่ถูกต้อง แต่ในโจทย์ที่มีข้อมูลขนาดใหญ่ แนวคิดนี้จะทำให้ใช้เวลานานและไม่ทันเวลาที่โจทย์กำหนด จึงต้องหาเทคนิคที่ดีกว่า หนึ่งในเทคนิคที่สำคัญและมีประสิทธิภาพสำหรับแก้ปัญหาหลายประเภท คือ กำหนดการพลวัติ หรือ Dynamic Programming (DP) นั่นเอง

1.1 Dynamic Programming คืออะไร

Dynamic Programming (DP) คือ เทคนิคการแก้ปัญหาโดยการแบ่งปัญหาใหญ่ออกเป็นปัญหาย่อย ๆ ที่ซ้ำกัน แล้วเก็บคำตอบของปัญหาย่อยเหล่านั้นไว้ใช้ซ้ำ แทนที่จะคำนวณใหม่ทุกครั้ง เทคนิคมีประโยชน์คือ ทำให้โปรแกรมที่ทำงานช้า เช่น ใช้เวลาเป็นเลขชี้กำลัง $O(2^n)$ ทำงานได้เร็วขึ้นมาก เช่น ใช้เวลาเป็นพหุนาม $O(n^2)$, $O(n^3)$ เป็นต้น เราจึงมักจะได้ยินบ่อยๆ ว่า Dynamic programming กับตาราง หรือ Bruteforce แบบฉลาด มีมุกที่มักจะกล่าวถึงเมื่อพูดถึง DP คือ

”Those who cannot remember the past are condemned to recompute it.”

โดยทั่วไปแล้ว DP จะใช้ได้กับปัญหาที่มีลักษณะ 2 อย่างคือ

- **Optimal Substructure (โครงสร้างย่อยที่เหมาะสมที่สุด)**: คำตอบที่ดีที่สุดของปัญหาใหญ่ขึ้นอยู่กับคำตอบที่ดีที่สุดของปัญหาย่อย ตัวอย่างเช่น การหาเส้นทางที่สั้นที่สุดจาก A ไปถึง C ซึ่งผ่าน B ถ้าเส้นทาง $A \to B \to C$ เป็นเส้นทางที่สั้นที่สุด ดังนั้นเส้นทาง $A \to B$ ก็ต้องเป็นเส้นทางที่สั้นที่สุดจาก A ถึง B ด้วย ส่วนนี้นำไปสู่การสร้าง Recurrence Relation

- **Overlapping Subproblems (ปัญหาย่อยที่ทับซ้อนกัน)**: ขั้นตอนการแก้ปัญหาจะต้องเรียกใช้ปัญหาย่อยเดิมซ้ำๆ หลายครั้ง ตัวอย่างเช่น การหาค่าลำดับที่ n ของเลขฟีโบนักชี สมมติชื่อฟังก์ชันว่า F แล้วต้องการหาค่า $F(5)$ จะต้องหา $F(4)$ และ $F(3)$ และในการหาค่า $F(4)$ ก็ต้องหา $F(3)$ และ $F(2)$ พบว่าปัญหาย่อย $F(3)$ จึงถูกเรียกใช้ซ้ำ ส่วนนี้นำไปสู่การใช้ Memoization หรือ Tabulation

1.2 รูปแบบของ Dynamic Programming

Dynamic Programming สามารถแบ่งแนวทางการใช้งานออกเป็น 2 รูปแบบหลัก ได้แก่ Top-down (Memoization) และ Bottom-up (Tabulation) โดยแต่ละแบบมีแนวคิดและข้อดี-ข้อจำกัดที่ต่างกัน ดังนี้

1.2.1 Top-down (Memoization)

Top-down หรือ Memoization เป็นการแก้ปัญหาที่เน้นการแบ่งปัญหาหลักออกเป็นปัญหาย่อย ๆ แล้วแก้ด้วย Recursive ทุกครั้งที่ฟังก์ชันแก้ปัญหาย่อยถูกเรียกจะตรวจสอบก่อนว่าเคยคำนวณผลลัพธ์ข้อนี้หรือยัง หากเคยแล้วจะนำคำตอบที่บันทึกไว้มาใช้งานทันที (โดยปกติจะเก็บไว้ใน Array หรือ Hash Map) จึงช่วยลดการคำนวณซ้ำซ้อนและเพิ่มประสิทธิภาพในการทำงาน

ข้อดี: โค้ดอ่านง่าย เข้าใจโครงสร้างปัญหาได้ดี เหมาะกับปัญหาที่นิยามแบบ Recursive ชัดเจน

ข้อจำกัด: หากขนาดปัญหาใหญ่, Recursive Calls อาจกิน Stack Space มากและเกิด Stack Overflow ได้

1.2.2 Bottom-up (Tabulation)

Bottom-up หรือ Tabulation เป็นการแก้ปัญหาโดยเริ่มจากปัญหาขนาดเล็กสุด (Base Case) แล้วค่อย ๆ คำนวณคำตอบของแต่ละปัญหาย่อยลงในตารางไล่ขึ้นมาตามลำดับจนถึงขนาดปัญหาที่ต้องการ ไม่ต้องใช้การเรียก Recursive ใช้การวนลูปเพื่อเติมตาราง

ข้อดี: ประหยัด Stack Memory กว่า Top-down มีประสิทธิภาพในการทำงานสูง และ Control Flow ชัดเจน

ข้อจำกัด: อาจต้องเตรียมและเติมตารางล่วงหน้าทั้งหมดแม้ส่วนหนึ่งไม่ได้ถูกใช้งาน โค้ดอาจอ่านยากกว่าในกรณีปัญหา Recursive ซับซ้อน

1.3 ตัวอย่างปัญหา Find the n-th Fibonacci number

ในส่วนนี้ขอยกตัวอย่างปัญหา Classic ผมเชื่อว่าปัญหานี้มักจะถูกยกมาเป็นตัวอย่างอันดับแรก ๆ ในการอธิบาย Dynamic Programming นั่นคือ ปัญหาลำดับที่ n ของเลขฟีโบนักชี หรือ Find the n-th Fibonacci number

โจทย์ข้อที่ 1: Find the n-th Fibonacci number

กำหนดจำนวนเต็มบวก n ให้หาค่าของเลขฟีโบนักชีลำดับที่ n

ลำดับฟีโบนักชี (Fibonacci Sequence) คือ ลำดับตัวเลขที่แต่ละตัว คือผลรวมของสองตัวก่อนหน้า โดยสองตัวแรกของลำดับฟีโบนักชี คือ 0 และ 1 ตัวอย่างลำดับฟีโบนักชี: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

ข้อมูลนำเข้า

ตัวเลขจำนวนเต็มบวก n 1 ตัว

ข้อมูลส่งออก

เลขฟีโบนักชีลำดับที่ n

ตัวอย่าง

ข้อมูลนำเข้า	ข้อมูลส่งออก
2	1
5	5

เรามาดูตัวอย่างวิธีการแก้ปัญหากัน เริ่มต้นด้วยการพิจารณาว่ามีลักษณะของปัญหาที่สามารถใช้ DP ได้ไหม

1. **Optimal Substructure (โครงสร้างย่อยที่เหมาะสมที่สุด)**

   หลักการของ Optimal Substructure คือ "คำตอบที่ดีที่สุด (หรือคำตอบที่ถูกต้อง) ของปัญหาใหญ่ สามารถสร้างขึ้นมาจากคำตอบที่ดีที่สุดของปัญหาย่อยๆ ได้"

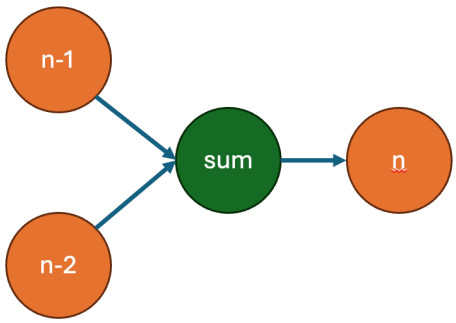
   ปัญหาลำดับที่ n ของเลขฟีโบนักชี สมมติว่าแทนด้วย $F(n)$ ปัญหาใหญ่คือการหา $F(n)$ ปัญหาย่อยคือการหา

$F(n-1)$ และ $F(n-2)$ จากนิยามของลำดับฟีโบนักชี เราทราบว่า $F(n) = F(n-1) + F(n-2)$ แสดงให้เห็นว่า คำตอบของ $F(n)$ นั้นขึ้นอยู่กับคำตอบของปัญหาย่อย $F(n-1)$ และ $F(n-2)$ โดยตรง ดังนั้น ปัญหาลำดับที่ n ของเลขฟีโบนักชีจึงมี Optimal Substructure

ทีนี้จากที่ผู้เขียนได้สอนมามีอีกวิธีช่วยในการหา Optimal Substructure คือการวาดรูป ซึ่งเราจะทำเริ่มต้นโดยเขียนวงกลมแทนค่าสุดท้ายหรือคำตอบที่ต้องการหาไว้ก่อน ดังภาพที่ 1.1

ภาพที่ 1.1: คำตอบสุดท้าย หรือค่าตอบลำดับที่ n

จากนั้นพิจารณาย้อนกลับไปว่า การที่จะคำนวณคำตอบที่ n ได้ ต้องอาศัยคำตอบของคำตอบใดบ้าง เราก็จะวาดวงกลมด้วยที่ภายในเป็นสถานะก่อนหน้าซึ่งอาจจะมีวงกลมคำตอบก่อนหน้าได้หลายอัน และคำตอบก่อนหน้าเหล่านี้มาทำการดำเนินการใดกันก็วาดด้วยวงกลมของการดำเนินการ ก่อนส่งผลที่ได้ไปให้กับคำตอบที่ n อย่างเช่น ในปัญหาลำดับที่ n ของเลขฟีโบนักชี เราจะพบว่าการหาค่าคำตอบลำดับที่ n ต้องใช้ค่าของคำตอบลำดับที่ $n-1$ และ $n-2$ มารวมกัน ดังนั้นจึงวาดภาพวงกลม $n-1$ และ $n-2$ จะมีลูกศรเชื่อมโยงจากวงกลมทั้งสองนี้มายังการดำเนินการรวม (Sum) จากนั้นชี้ผลลัพธ์ไปที่วงกลมที่ n ดังภาพที่ 1.2

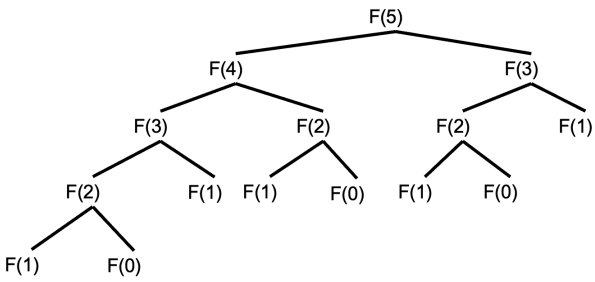


ภาพที่ 1.2: คำตอบลำดับที่ n เกิดจากการรวมกันของคำตอบลำดับที่ $n-1$ และ $n-2$

2. **Overlapping Subproblems (ปัญหาย่อยที่ซ้ำซ้อน)**

   หลักการของ Overlapping Subproblems คือ “ในกระบวนการแก้ปัญหา จะมีปัญหาย่อยบางอย่างที่ถูกเรียกใช้ซ้ำหลายครั้ง”

   ในปัญหาลำดับที่ n ของเลขฟีโบนักชีมีจุดที่ทำให้การแก้ปัญหาแบบธรรมดา (Recursive ธรรมดา) มีประสิทธิภาพต่ำมาก ลองมาดูตัวอย่างการคำนวณ $F(5)$ แบบ Recursive ธรรมดา ดังภาพที่ 1.3 จะเห็นได้ว่า $F(3)$ ถูกคำนวณ 2 ครั้ง, $F(2)$ ถูกคำนวณ 3 ครั้ง และ $F(1)$ ถูกคำนวณ 5 ครั้ง การคำนวณซ้ำๆ นี้คือ Overlapping Subproblems นั่นเอง ซึ่งถ้าเราคำนวณแล้วเก็บผลลัพธ์ไว้ก็จะได้ไม่ต้องมาคำนวณใหม่



ภาพที่ 1.3: ตัวอย่างการคำนวณ $F(5)$

พอเห็นว่ามีทั้ง 2 เงื่อนไขแล้ว เราจะมากำหนด **State** ให้กับปัญหา State คือชุดหรือจำนวนของพารามิเตอร์ที่ใช้ในการระบุปัญหาย่อยแต่ละปัญหาอย่างไม่ซ้ำกัน ในการแก้ปัญหาด้วย DP เราจะเก็บคำตอบของแต่ละ State ซึ่งเป็นคำตอบของปัญหาย่อยไว้ในตาราง ดังนั้น

- State ต้องมีข้อมูลเพียงพอที่จะอธิบายสถานะปัญหาย่อยนั้น ๆ ได้อย่างสมบูรณ์
- State ควรจะมีขนาดเล็กและไม่ซับซ้อนเกินความจำเป็น
- State ของปัญหาย่อยมาสร้างเป็น State ของปัญหาที่ใหญ่ขึ้นได้

สำหรับปัญหาลำดับที่ n ของเลขฟีโบนักชี ปัญหาย่อยแต่ละอันสามารถระบุได้ด้วยพารามิเตอร์หรือตัวเลขเพียงตัวเดียว นั่นคือ ตำแหน่งในลำดับ (n) ดังนั้น State ของเราก็คือตัวเลข n นั่นเอง เราสามารถนิยามตารางของเราได้ดังนี้ $dp[i]$ หมายถึง "ค่าของเลขฟีโบนักชีลำดับที่ i" หรือก็คือคำตอบของปัญหาย่อยที่กำหนดโดย State i การกำหนด State ให้ชัดเจนนี้เองที่ทำให้เราสามารถเขียน Recurrence Relation ได้อย่างถูกต้อง ทั้งนี้แต่ละปัญหาการกำหนด State อาจจะไม่เหมือนกัน บางปัญหาต้องใช้หลายตัวแปรเพื่อระบุปัญหาย่อย ซึ่งจะมีตัวอย่างให้ดูในปัญหาต่อ ๆ ไป

ต่อมา เราจะมาเขียนสมการเวียนเกิด หรือ Recurrence Relation สมการเวียนเกิดคือสูตรคณิตศาสตร์ที่อธิบายความสัมพันธ์ระหว่างปัญหาใหญ่และปัญหาย่อย ประกอบด้วย

- **กรณีฐาน หรือ Base Case** คือส่วนของสมการเวียนเกิดที่กำหนดจุดเริ่มต้นของลำดับ เป็นจุดที่หยุดการเรียกตัวเองซ้ำ ๆ ของสมการเวียนเกิด หรือมองเป็นส่วนที่คืนค่าได้เลยก็ได้
- **กรณีเวียนเกิด หรือ Recursive Case** คือส่วนของสมการเวียนเกิดที่นิยามค่าของของฟังก์ชันที่มีพารามิเตอร์ n โดยอาศัยค่าพารามิเตอร์ที่น้อยกว่า n หรือมองว่าเป็นส่วนของการเรียกตัวเองนั่นเอง ซึ่งเราจะนำเอาความสัมพันธ์หรือรูปภาพจากการวิเคราะห์ Optimal Substructues มาใช้ ตัวอย่างเช่นภาพ 1.2 หากเราสมมติว่าตั้งชื่อ State เป็นฟังก์ชัน F แล้วแปลงรูป ก็จะได้ว่า วงกลมด้านซ้าย n คือ $F(n)$ เกิดมาจาก $F(n+1) + F(n+2)$

ดังนั้นสำหรับปัญหาลำดับที่ n ของเลขฟีโบนักชี คือ

$$F(n) = \begin{cases} 0, & \text{ถ้า } n = 0 \\ 1, & \text{ถ้า } n = 1 \\ F(n-1) + F(n-2), & \text{ถ้า } n > 1 \end{cases}$$

นั่นคือ กรณีฐาน หรือ Base Cases คือ $n = 0$ และ $n = 1$ ส่วน Recursive Case คือ $n > 1$

ต่อมาเป็นส่วนที่เขียนโค้ด เราจะค่อยๆ พัฒนาโค้ดทีละขั้นตอน ตอนนี้เรามี Recurrence Relation แล้ว วิธีแก้แบบทั่วไปก็แปลง Recurrence Relation ไปเป็นโค้ดโดยตรง จะได้ Recursive Solution ตรงส่วนไหนที่เป็น Base Case เราก็เขียนเงื่อนไขของ Base Case แล้ว return คำตอบเลย ส่วน Recursive Case เราก็ return เป็นการเรียกฟังก์ชัน แทบจะเรียกว่าเขียนโค้ดได้ตรง ๆ จาก Recurrence Relation ในโค้ดจะเปลี่ยนฟังก์ชัน F เป็น $fib_recursive$ เพื่อความชัดเจน

```cpp
#include <bits/stdc++.h>
using namespace std;
long long fib_recursive(int n) {
    // Base cases
    if (n <= 1) {
        return n;
    }
    // Recursive case
    return fib_recursive(n - 1) + fib_recursive(n - 2);
}
int main() {
    int n;
    cin>>n;
    cout << fib_recursive(n) << "\n";
    return 0;
}
```

เมื่อเรียก $fib_recursive(n)$ แล้วมักจะเรียก $fib_recursive(n-1)$ และ $fib_recursive(n-2)$ ซ้ำไปเรื่อย ๆ ซึ่งในการเรียกแต่ละครั้งจะเกิดการแตกของคำสั่งเป็น 2 ส่วน (ยกเว้นกรณีฐาน) ดังนั้นเวลาในการทำงานจึงเป็น $O(2^n)$ ซึ่งวิธีนี้เวลาในการทำงานของโค้ดนี้สูงมาก

ขั้นต่อมา Top-down DP หรือ Memoization วิธีนี้จะปรับปรุงจาก Recursive Solution โดยเพิ่ม "ความจำ" เข้าไป สังเกตได้จาก Overlapping Subproblems ในภาพ 1.3 พบว่ามีการการซ้ำของการเรียกฟังก์ชันให้คำนวณด้วย พารามิเตอร์ค่าเดิม ดังนั้นเราจะใช้โครงสร้างข้อมูลแบบ Array ที่มีจำนวนมิติสอดคล้องกับจำนวนพารามิเตอร์มาใช้ในการเก็บผลลัพธ์ที่คำนวณไปแล้วซึ่งคือค่าของแต่ละ State นั่นเอง วิธีการก็คือ สร้างตาราง 1 มิติตามจำนวนพารามิเตอร์เพื่อเก็บค่า กำหนดค่าเริ่มต้นของตารางเป็นค่าพิเศษ (เช่น 0 หรือ -1) ที่เป็นการบอกว่า State นี้ยังไม่เคยคำนวณ ก่อนจะคำนวณคำตอบให้ตรวจสอบก่อนว่าในตารางเก็บค่าที่ไม่ใช่ค่าเริ่มต้นอยู่หรือไม่ ถ้าเก็บแสดงว่าเคยคำนวณแล้วดังนั้นให้คืนค่าในตารางเพื่อจะได้ไม่ต้องคำนวณซ้ำ ถ้าไม่ได้เป็นค่าเริ่มต้นให้คำนวณตามปกติ แต่พอคำนวณได้ผลลัพธ์แล้ว ให้เก็บผลลัพธ์นั้นลงในตารางก่อนแล้วค่อยคืนค่า

```
#include <bits/stdc++.h>
using namespace std;
vector<long long> memo;
long long fib_top_down(int n) {
    // check memo
    if (memo[n] != -1) {
        return memo[n];
    }
    // Base cases
    if (n <= 1) {
        return n;
    }
    // Compute and store the values in a table
    long long result = fib_top_down(n - 1) + fib_top_down(n - 2);
    memo[n] = result;
    return result;
}
int main() {
    int n;
    cin>>n;
    memo.resize(n + 1, -1); // Resize and set the initial value
    cout << fib_top_down(n) << "\n";
    return 0;
}
```

วิธีนี้ทำงานเร็วกว่าวิธี Recursive Solution เนื่องจากแก้ปัญหา Overlapping Subproblems ได้ Time Complexity เป็น $O(n)$ เพราะแต่ละ State $F(k)$ สำหรับ k จาก 0 ถึง n จะถูกคำนวณเพียงครั้งเดียว

มาถึง DP รูปแบบสุดท้าย Bottom-up DP หรือ Tabulation วิธีนี้จะเริ่มจากสร้างคำตอบของปัญหาย่อยที่เล็กที่สุด (Base Cases) แล้วนำเอาคำตอบย่อยที่มีค่อยๆ สร้างคำตอบไปจนถึงปัญหาใหญ่ที่เราต้องการ วิธีการคือ สร้างตาราง dp ขนาด $n+1$ แล้วกำหนดค่าเริ่มต้นซึ่งคือค่าของ Base Cases ให้กับตาราง ในกรณีนี้คือ $dp[0] = 0$ และ $dp[1] = 1$ จากนั้นวนลูปถัดจาก Base Cases ในที่นี้คือจาก $i = 2$ ถึง n ในแต่ละรอบของลูปจะคำนวณ $dp[i]$ (คำตอบของ State i) จากค่าของ State อื่นที่เรามีอยู่ก่อนแล้ว ในที่นี้คือ $dp[i] = dp[i-1] + dp[i-2]$ ส่วนนี้สามารถทำงานได้เพราะว่าค่าของ $dp[i-1]$ และ $dp[i-2]$ มีการคำนวณค่าไว้แล้ว ซึ่งในส่วนนี้ก็คือ Recursive Case ของ Recurrence Relation นั่นเอง หลังจากวนลูปเสร็จคำตอบของปัญหาคือ $dp[n]$ ทั้งนี้ในบางปัญหาอาจจะต้องกำหนดค่าเริ่มต้นให้มากพอเพื่อที่จะทำให้การคำนวณในลูปทำงานได้

```
#include <bits/stdc++.h>
using namespace std;
long long fib_bottom_up(int n) {
    // Manage special case
    if (n <= 1) {
        return n;
    }
    // Create a dp table to store the value of each state
    vector<long long> dp(n + 1);
    // Base Cases
    dp[0] = 0;
```

```
    dp[1] = 1;
    // Build the solution from smaller subproblems to larger ones
    for (int i = 2; i <= n; ++i) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    // Return final answer
    return dp[n];
}
int main() {
    int n;
    cin>>n;
    cout << fib_bottom_up(n) << "\n";
    return 0;
}
```

เวลาในการทำงานเท่ากับแบบ Top-down DP และมักจะใช้หน่วยความจำน้อยกว่า (เพราะไม่มีค่าใช้จ่ายของ Call Stack) Time Complexity เป็น $O(n)$ เนื่องจากมีการวนลูป n รอบ

นอกจากนี้หากพิจารณาดี ๆ แล้วจะพบว่าเราใช้ค่าก่อนหน้าเพียงสองค่าในการคำนวณ ดังนั้นไม่จำเป็นต้องเก็บทุกค่าทั้ง Array ก็ได้ ทำให้สามารถปรับโค้ดเพื่อประหยัดหน่วยความจำได้ดังโค้ดต่อไปนี้

```
#include <bits/stdc++.h>
using namespace std;
long long fib_bottom_up_optimized(int n) {
    // Manage special case
    if (n <= 1) {
        return n;
    }
    // Store only the last two computed values
    long long prev2 = 0; // F(0)
    long long prev1 = 1; // F(1)
    long long current = 0;

    for (int i = 2; i <= n; ++i) {
        current = prev1 + prev2; // F(i) = F(i-1) + F(i-2)
        prev2 = prev1;           // Update old value
        prev1 = current;         // Update new value
    }
    return current; // At the end of the loop, current contains the value of F(n).
}
int main() {
    int n;
    cin>>n;
    cout << fib_bottom_up_optimized(n) << "\n";
    return 0;
}
```

1.4 ขั้นตอนการแก้ปัญหาแบบ DP

จากตัวอย่างโจทย์ข้อที่ 1 เมื่อเจอโจทย์หรือปัญหาอื่นๆ เราก็ควรตรวจสอบดังนี้

1. **ตรวจสอบว่าเป็นปัญหา DP หรือไม่** ทำได้โดยลองแบ่งปัญหาเป็นปัญหาย่อย ดูว่ามี Optimal Substructure และมี Overlapping Subproblems หรือไม่ หรืออาจจะดูแนวว่าเป็นการถามหาค่าที่ดีที่สุด หรือการนับจำนวนบางอย่าง

2. **กำหนด State (สถานะ) ของ DP** นี่คือส่วนที่ยากที่สุดและสำคัญที่สุด เพราะว่าส่วนนี้แหละเป็นส่วนที่จะตัดสินใจว่าจะเก็บข้อมูลอะไรบ้างในตาราง DP หรือเขียนว่า $dp[i]$ หมายถึงอะไร (เช่น คำตอบที่ดีที่สุดสำหรับปัญหาขนาด i) ทั้งนี้ State ต้องครอบคลุมข้อมูลพอที่จะคำนวณคำตอบของ State ถัดไปได้

3. **หาสมการ Recurrence Relation (สมการเวียนเกิด)** หาความสัมพันธ์ระหว่าง State ต่างๆ จะหาค่า $dp[i]$ จาก $dp[i-1]$, $dp[i-2]$ หรือ State อื่นๆ ได้อย่างไร นี่คือสำคัญของของอัลกอริทึม กำหนด Base Cases (กรณีฐาน):

ค่าเริ่มต้นของตาราง DP ที่เรารู้คำตอบโดยไม่ต้องคำนวณ (เช่น $dp[0]$, $dp[1]$) ส่วนใหญ่ที่พลาดหรือไม่สามารถพัฒนาต่อได้ คือข้ามการเขียน Recurrence Relation ไป จะไปจำตาราง จำ pattern เสียมากกว่า

4. **เขียนโค้ด** นำเอา Recurrence Relation มาแปลงเป็นโค้ด โดยเขียนเป็น Recursive Solution จากนั้นปรับเป็น Top-down Solution หรือ Bottom-up Solution ต่อไป

1.5 กลุ่มการแก้ปัญหาด้วย DP

ในหนังสือเล่มนี้ ผู้เขียนได้จัดกลุ่มการแก้ปัญหาที่น่าสนใจ 10 กลุ่มที่แก้ด้วยกำหนดการพลวัติที่คล้ายๆ กันรวมกันไว้เป็นหัวข้อเดียวกันเพื่อความสะดวกในการเรียนรู้ และช่วยในการวิเคราะห์ปัญหาดังนี้

1. **Linear DP**: ปัญหา DP ที่สถานะสามารถแสดงด้วยการเคลื่อนที่เป็นเส้นตรง เช่น การหาค่าที่ดีที่สุดตามลำดับตัวเลข การแลกเหรียญ
2. **Knapsack DP**: ปัญหาที่เกี่ยวข้องกับการเลือกสิ่งของหลายชิ้นให้ได้ค่าสูงสุดโดยมีข้อจำกัด เช่น น้ำหนักหรือจำนวน
3. **Path Finding on Grids**: การหาวิถี (Path) ที่ดีที่สุดบนกริดสองมิติ เช่น ตารางหรือแผนที่
4. **Sequence DP**: การแก้ปัญหาจัดเรียงหรือเปรียบเทียบลำดับข้อมูล เช่น Longest Increasing Subsequence, Edit Distance
5. **Tiling DP**: การปูกระเบื้องเป็นการแก้ปัญหาวางกระเบื้องรูปแบบต่าง ๆ ให้เต็มพื้นที่
6. **DP on Trees**: การแก้ปัญหาบนโครงสร้างข้อมูลแบบต้นไม้ (Tree Structure) เช่น การหาค่าสูงสุด/ต่ำสุดใต้ต้นไม้แต่ละโหนด
7. **State Compression (Bitmask DP)**: การใช้ Bitmask เพื่อบีบอัดสถานะที่มีความซับซ้อนสูงให้ใช้พื้นที่เก็บข้อมูลและเวลาที่เหมาะสม
8. **Digit DP**: เทคนิค DP สำหรับแก้ปัญหาที่เกี่ยวกับค่าหลัก (Digit) ของจำนวนเต็ม เช่น การนับจำนวนตัวเลขที่มีคุณสมบัติบางอย่างในช่วงที่กำหนด
9. **Interval DP**: การแก้ปัญหารูปแบบช่วง (ช่วงตัวแปรเริ่มต้นและสิ้นสุด) เช่น การแบ่งย่อยช่วงหรือการจับคู่ข้อมูลในช่วง
10. **DP on Graph**: การแก้ปัญหา DP ที่ต้องประยุกต์กับกราฟ เช่น การนับเส้นทางในกราฟ หรือหาค่าใช้งานในกราฟที่มีเงื่อนไขพิเศษ

1.6 สิ่งที่อยากฝากไว้ก่อนอ่านบทต่อไป

จากประสบการณ์ของผู้เขียนในการสอนคอมพิวเตอร์โอลิมปิกและสอนวิชาอัลกอริทึมพบว่า ปัญหาสำคัญในการเรียนรู้เรื่อง Dynamic Programming คือ ผู้เรียนมักจะข้ามขั้นตอน “การคิด Recurrence Relation” แล้วหันไปจำแค่ “รูปแบบตาราง” เช่น การเอาค่าในตารางตำแหน่งบนหรือตัวก่อนหน้ามาบวกเข้าด้วยกันโดยไม่เข้าใจหลักการเบื้องหลัง

การ “จำตาราง” หรือ “จำแพทเทิร์น” ในลักษณะนี้อาจช่วยให้ทำโจทย์บางข้อได้ชั่วคราว แต่จะส่งผลต่อความเข้าใจในระยะยาว เพราะจริงๆ แล้วแต่ละปัญหามีลักษณะเฉพาะและข้อแตกต่างที่สำคัญ หากไม่เข้าใจโครงสร้างของปัญหาจริง ๆ จะทำให้ไม่สามารถพลิกแพลงหรือประยุกต์แก้ปัญหาใหม่ ๆ ได้

เพื่อแก้ไขปัญหานี้ ผู้เขียนจึงตั้งใจหลังโจทย์จะเว้นที่ไปยังหน้าถัดไปเพื่อให้คิดก่อนและจะแทรกเนื้อหา Recurrence Relation, Top-down DP และ Bottom-up DP ไว้ในแต่ละตัวอย่างตามลำดับ จึงขอแนะนำให้ผู้อ่านเมื่อเจอโจทย์ให้อ่านทำความเข้าใจ จากนั้น “ลองคิด Recurrence Relation ด้วยตัวเองก่อน” แล้วจึงศึกษาวิธีเขียนโค้ดในรูปแบบ Top-down และ Bottom-up DP ตามลำดับ

บทที่ 2

Linear DP

ในหัวข้อแรกที่เราจะมาพิจารณากันคือ Linear DP ซึ่งเป็นรูปแบบ DP ที่เป็นพื้นฐานและพบเห็นได้บ่อยที่สุด Linear DP คือกลุ่มปัญหา DP ที่สถานะปัญหาปัจจุบัน ($dp[i]$) เกิดจากผลลัพธ์ของปัญหาย่อยที่มาก่อนหน้าเท่านั้น (เช่น $dp[i-1]$, $dp[i-2]$, $dp[i-k]$, ...) ลักษณะเด่นคือเราสามารถแก้ปัญหาได้โดยการเรียงลำดับการคำนวณจากปัญหาย่อยที่เล็กที่สุดไปหาปัญหาย่อยที่ใหญ่ขึ้นเรื่อย ๆ ในลำดับแนวตรง (เช่น จาก $dp[0]$ ไป $dp[1]$, $dp[2]$, ..., $dp[n]$)

ในบทนี้เราจะมีดูปัญหา Climbing Stairs, The Frog Jump Problem: Counting Ordered Ways, Coin Change: Minimum Coins, House Robber, Maximum Subarray Sum และ Prefix Sum

2.1 Climbing Stairs

ลองจินตนาการว่าเรากำลังยืนอยู่หน้าบันไดที่มีขั้นอยู่จำนวนหนึ่ง ซึ่งเป็นสิ่งที่เราพบเจอและทำกันทุกวัน แต่วันนี้ มีกฎพิเศษเข้ามาคือ เราสามารถเลือกที่จะก้าวขึ้นได้ทีละ 1 ขั้น หรือกระโดดข้ามไปทีละ 3 ขั้น คำถามคือมีทางเดินที่แตกต่างกันทั้งหมดกี่วิธีที่จะไปให้ถึงบันไดขั้นสุดท้าย

ในตอนแรก ปัญหานี้อาจดูเหมือนเกมง่าย ๆ ที่เด็ก ๆ ชอบเล่น แต่เมื่อจำนวนขั้นบันไดเพิ่มขึ้นจาก 4, 5, ไปจนถึง 20 ขั้น เราจะพบว่าจำนวนวิธีที่เป็นไปได้นั้นเพิ่มขึ้นอย่างมหาศาลและไม่สามารถนับได้ง่าย ๆ การพยายามไล่ลิสต์ทุกวิธีจะกลายเป็นเรื่องที่เป็นไปไม่ได้

2.1.1 รูปแบบเป็นทางการของปัญหา

โจทย์ข้อที่ 2: Climbing Stairs

คุณกำลังเดินขึ้นบันได โดยคุณสามารถเดินขึ้นได้ทีละ 1 ขั้น หรือ 3 ขั้น จงหาจำนวนวิธีทั้งหมดที่เป็นไปได้ในการเดินขึ้นบันไดถึงขั้นที่ n

ข้อมูลนำเข้า
ตัวเลขจำนวนเต็มบวก n 1 ตัว แทนขั้นที่ต้องการไปถึง

ข้อมูลส่งออก
จำนวนวิธีทั้งหมดในการปีนบันไดถึงขั้นที่ n

ตัวอย่าง

ข้อมูลนำเข้า	ข้อมูลส่งออก
4	3
10	28

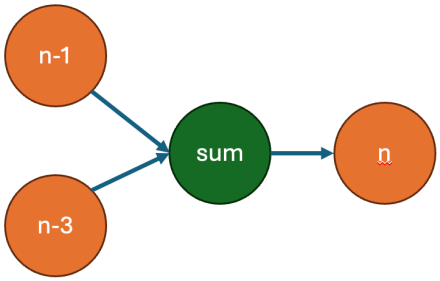
ลองคิดดูก่อนเปิดหน้าถัดไป ข้อแนะนำก่อนจะทำอะไร ให้ลองยกตัวอย่างด้วยข้อมูลนำเข้าเล็กๆ เช่น 1, 2, 3 ก่อนเพื่อดูว่าหากข้อมูลเข้าเล็ก ๆ จะคำนวณคำตอบได้อย่างไร

2.1.2 วิเคราะห์ว่าสามารถใช้ DP ในการแก้

ในวิเคราะห์ DP กัน ก่อนอื่นเรามาดูคุณสมบัติของปัญหา Climbing Stairs กันก่อน ได้แก่ Optimal Substructure และ Overlapping Subproblems โดยในส่วน Optimal Substructure หากลองพิจารณาเป็นภาพวงกลม จะได้ว่า การที่จะมาถึงขั้นที่ n (หรือ วงกลม n) มีรูปแบบที่เกิดขึ้นได้โดยพิจารณาก้าวสุดท้าย

- หากก้าวสุดท้ายด้วย 1 ขั้น แสดงว่าเรามาจากขั้นที่ $n-1$ เราก็เขียนวงกลม $n-1$
- หากก้าวสุดท้ายด้วย 3 ขั้น แสดงว่าเรามาจากขั้นที่ $n-3$ เราก็เขียนวงกลม $n-3$

ซึ่งเรานับรวมทั้ง 2 รูปแบบ ดังนั้นวงกลม $n-1$ และวงกลม $n-3$ ก็มาชี้ที่การดำเนินการ “รวม” แล้วจึงชี้ไปที่วงกลม n ดังภาพที่ 2.1



ภาพที่ 2.1: คำตอบของขั้นที่ n เกิดจากการรวมกันของคำตอบขั้นที่ $n-1$ และ $n-3$

ในส่วน Overlapping Subproblems เมื่อใช้แนวทางแบบ Recursive ในการหาคำตอบสมมติว่าเราเรียกทำงานด้วยฟังก์ชัน climb จะพบว่ามีการคำนวณค่าของปัญหาย่อยเดิม ๆ ซ้ำหลายครั้ง เช่น ในกรณีของ climb(5) จะต้องเรียก climb(4) และ climb(2) ซึ่งเมื่อลงลึกไปอีก การคำนวณ climb(4) ก็จะต้องเรียก climb(3) และ climb(1) และการคำนวณ climb(3) ยังต้องเรียก climb(2) และ climb(0) ทำให้ climb(2) ถูกเรียกคำนวณซ้ำโดยไม่จำเป็น หากค่า n มีขนาดใหญ่ขึ้น จำนวนครั้งที่ปัญหาย่อยซ้ำกันจะเพิ่มขึ้นมาก

2.1.3 ขั้นตอนการแก้ปัญหาแบบ DP

การสร้าง State

สมมติว่าตั้งชื่อ State ของปัญหานี้ว่า climb จะได้ว่า climb(i) = จำนวนวิธีทั้งหมดในการปีนบันไดถึงขั้นที่ i หากสังเกตดีๆ ปัญหานี้คล้ายกับปัญหาที่ 1 ลำดับที่ n ของเลขฟีโบนักชีเลย เพียงแต่เปลี่ยนจากเอาค่าจาก State ที่ $n-2$ เป็น State ที่ $n-3$ แทน

Recurrence Relation

เมื่อนำส่วน Optimal Substructure มาพิจารณาความสัมพันธ์ของ State ในส่วนเรียกตัวเองใน Recurrence Relation ได้เลย จะได้ว่า climb(n) = climb$(n-1)$ + climb$(n-3)$ หลังจากนั้นก็คิดว่าส่วน Base Cases เป็นอะไรบ้าง พบว่า ถ้าขั้นน้อยกว่า 3 ก็เดินได้แบบเดียวมากจากการก้าว 1 ขั้น

$$\text{climb}(n) = \begin{cases} 1, & \text{ถ้า } n < 3 \\ \text{climb}(n-1) + \text{climb}(n-3), & \text{ถ้า } n \geq 3 \end{cases}$$

Recursive Solution

เป็นการนำเอา Recurrence Relation มาเขียนเป็นโค้ดโดยตรง โดยเขียนส่วนเรียก Base Cases ด้วยการคืนค่าเลย และส่วน Recursive Case ก็คืนค่าเป็นการเรียกฟังก์ชัน เช่นเดียวกับ ปัญหาลำดับที่ n ของเลขฟีโบนักชี

```
long long climb_recursive(int n) {
    if (n == 0 || n == 1 || n == 2) { // Base Cases
        return 1;
    }
    // Recursive Case
    return climb_recursive(n - 1) + climb_recursive(n - 3);
}
```

เวลาในการทำงานเป็น $O(2^n)$

Top-down Solution

ทำการเพิ่ม memo ให้กับ Recursive Solution จะได้

```
long long climb_memo(int n) {
    // check memo
    if (memo[n] != -1) {
        return memo[n];
    }
    if (n == 0 || n == 1 || n == 2) { // Base Cases
        return 1;
    }
    // Compute and store the values in a table
    long long result = climb_memo(n - 1) + climb_memo(n - 3);
    memo[n] = result;
    return result;
}
```

เวลาในการทำงานเป็น $O(n)$

Bottom-up Solution

เราจะเพิ่มค่า Base Cases ลงใน array ก่อนหลังจากนั้นวนรอบเรียกใช้โครงสร้าง Recursive Case ตั้งแต่ Base Cases จนตำแหน่งสุดท้าย

```
long long climb_bottom_up(int n) {
    if (n == 0 || n == 1 || n == 2) {
        return 1;
    }
    //Create a table of size n+1
    vector<long long> dp(n + 1);
    dp[0] = dp[1] = dp[2] = 1; //Base Cases
    //Calculate from the 3rd step up to
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }
    return dp[n];
}
```

เวลาในการทำงานเป็น $O(n)$ ทั้งนี้สามารถประหยัดหน่วยความจำได้เช่นเดียวกับปัญหา ลำดับที่ n ของเลขฟีโบนักชี

```
long long climb_optimized(int n) {
    if (n == 0 || n == 1 || n == 2) {
        return 1;
    }
    long long prev3 = prev2 = prev1 = 1; // f(0) f(1) f(2)
    long long current;
    for (int i = 3; i <= n; i++) {
```

```
        current = prev1 + prev3;
        prev3 = prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

เวลาในการทำงานเป็น $O(n)$

2.2 The Frog Jump Problem: Counting Ordered Ways

สมมติว่ามีกบตัวหนึ่ง มันยืนอยู่ที่จุดเริ่มต้นของเส้นทาง (ตำแหน่งที่ 0) เป้าหมายของมันคือการไปถึงใบบัวที่อยู่ในตำแหน่งที่ n แต่นี่ไม่ใช่กบธรรมดา มันสามารถกระโดดไปข้างหน้าได้ 1, 3 หรือ 5 หน่วยในการกระโดดแต่ละครั้ง

คำถามในข้อนี้ไม่ใช่ "มันจะไปถึงได้หรือไม่" แต่ว่า "มันมี **ลำดับการกระโดด** ที่แตกต่างกันทั้งหมดกี่แบบในการไปถึงเป้าหมาย"

ลองพิจารณาสมมติเป้าหมายที่ตำแหน่ง 4:

- มันอาจกระโดด 1 หน่วย 4 ครั้ง (1, 1, 1, 1)
- หรือกระโดด 1 หน่วย แล้วตามด้วย 3 หน่วย (1, 3)
- หรือกระโดด 3 หน่วย แล้วตามด้วย 1 หน่วย (3, 1)

แม้ว่าคำตอบแบบที่สองและสามจะใช้จำนวนครั้งในการกระโดดเท่ากัน แต่รูปแบบเส้นทางที่ได้แตกต่างกัน ปัญหานี้ช่วยให้คิดในเชิง "ลำดับ" และ "เส้นทาง" ทำให้เราระวังเกี่ยวกับความแตกต่างระหว่างการนับชุด (Combinations) กับการนับลำดับ (Permutations)

2.2.1 รูปแบบเป็นทางการของปัญหา

โจทย์ข้อที่ 3: The Frog Jump Problem

มีกบตัวหนึ่งอยู่ที่จุดเริ่มต้น (จุด 0) บนเส้นจำนวน และต้องการไปยังจุดเป้าหมายที่ตำแหน่ง n กบสามารถกระโดดไปข้างหน้าได้เพียง 1 หน่วย, 3 หน่วย หรือ 5 หน่วยในแต่ละครั้ง จงหาจำนวนลำดับการกระโดดที่แตกต่างกันทั้งหมดที่จะทำให้กบไปถึงจุดเป้าหมายได้

ข้อมูลนำเข้า
ตัวเลขจำนวนเต็มบวก n 1 ตัว แทนตำแหน่งเป้าหมายบนเส้นจำนวน

ข้อมูลส่งออก
จำนวนลำดับการกระโดดทั้งหมดที่เป็นไปได้ในการไปถึงจุดเป้าหมายที่ตำแหน่ง n

ตัวอย่าง

ข้อมูลนำเข้า	ข้อมูลส่งออก
5	5
10	47

คำอธิบายตัวอย่าง:
สำหรับตัวอย่างแรก, $n = 5$
ลำดับการกระโดดที่เป็นไปได้:

- 1, 1, 1, 1, 1
- 3, 1, 1
- 1, 3, 1
- 1, 1, 3
- 5

รวมเป็น 5 ลำดับ

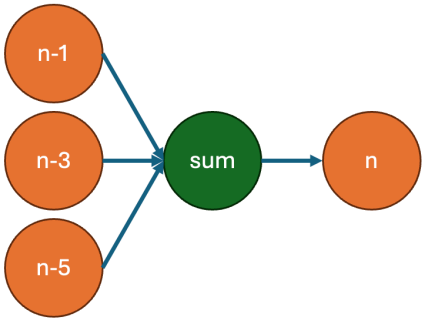
ลองคิดดูก่อนเปิดหน้าถัดไป

2.2.2 วิเคราะห์ว่าสามารถใช้ DP ในการแก้

เริ่มต้นเรามาวิเคราะห์ Optimal Substructure กันก่อน หากใช้วิธีวาดรูปวงกลม ลองนึกว่าที่ตำแหน่ง n เราก็วาดรูปวงกลมไว้ แล้วถามย้อนกลับไปว่าการที่จะมายังตำแหน่งที่ n นั้นมาจากทางใดได้บ้าง จะได้ว่าเราสามารถกระโดดครั้งสุดท้ายด้วยระยะ 1, 3, 5 หน่วยแล้วมาถึงตำแหน่งที่ n ได้ นั่นคือมาจากตำแหน่ง $n-1, n-3, n-5$ ดังนั้นการหาวิธีมาถึงตำแหน่ง n จะเท่ากับการนำจำนวนวิธีที่มาถึงแต่ละตำแหน่งก่อนหน้า (ที่กระโดดมาถึง n ได้) มารวมกัน สรุปคือ

- ถ้ากระโดด 1 หน่วยจาก $n-1$ มาถึง n ก็ใช้คำตอบของตำแหน่ง $(n-1)$
- ถ้ากระโดด 3 หน่วยจาก $n-3$ มาถึง n ก็ใช้คำตอบของตำแหน่ง $(n-3)$
- ถ้ากระโดด 5 หน่วยจาก $n-5$ มาถึง n ก็ใช้คำตอบของตำแหน่ง $(n-5)$

สรุปคือวิธีการไปถึงตำแหน่ง n คือผลรวมของวิธีการที่ไปถึง $n-1$, $n-3$, และ $n-5$ ดังภาพที่ 2.2



ภาพที่ 2.2: คำตอบของตำแหน่งที่ n เกิดจากการรวมกันของคำตอบตำแหน่งที่ $n-1$ และ $n-3$ และ $n-5$

ในส่วน Overlapping Subproblems นั้น สมมติว่าเราเรียกทำงานด้วยฟังก์ชัน ways เมื่อพิจารณาตัวอย่างหาจำนวนลำดับที่ไปถึงขั้น 6 โดยกระโดด 1, 3, 5 ขั้น นั้น

- ways(6) จะเรียก ways(5) (กระโดด 1 ขั้นมาถึง), ways(3) (กระโดด 3 ขั้นมาถึง), และ ways(1) (กระโดด 5 ขั้นมาถึง)
- ส่วน ways(5) จะเรียก ways(4), ways(2) และ ways(0)
- ส่วน ways(4) จะเรียก ways(3), ways(1) และ ways(−1)
- เรียกต่อไป ...

ทั้งนี้ ways(3) จะถูกเรียกโดยตรงจาก ways(6) และอีกครั้งจาก ways(4) จะเห็นได้ว่า ways(3) ถูกเรียกคำนวณซ้ำ ปัญหาย่อยเหล่านี้จะถูกคำนวณซ้ำจำนวนมากนั่นเอง

2.2.3 ขั้นตอนการแก้ปัญหาแบบ DP

การสร้าง State

"State" หรือข้อมูลที่เราต้องการจดจำเพื่อแก้ปัญหา ในที่นี้ State ที่เราสนใจคือ: ways(i) = จำนวนวิธีที่กบจะไปถึงตำแหน่งที่ i ได้ สังเกตตัวอย่างนี้คล้ายกับโจทย์ปัญหาลำดับที่ n ของเลขฟีโบนักชี และ Climbing Stairs ก่อนหน้าเลยเพียงแต่ว่ามี State ก่อนหน้าที่ต้องใช้มากกว่า

Recurrence Relation

หากเรานำส่วนของ Optimal Substructure มาเขียนเป็นสมการจะได้ว่า

$$\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-3) + \text{ways}(n-5)$$

สิ่งที่ต้องทำต่อมาคือคิด Base Case ซึ่งเป็น Case เริ่มต้นที่ทำให้ส่วน Recursive เป็นจริง มักจะเป็น Case 0, Case ขอบ หรือ Case ติดลบ ดังนั้น:

- ways(0) = 1 หมายถึง กบอยู่ตำแหน่ง 0 มี 1 วิธี คือ ยังไม่ได้กระโดดเลย และ
- ways(i) = 0 สำหรับ $i < 0$ เพราะว่าเป็นกรณีเป็นไปไม่ได้

เมื่อรวมทั้ง Base Case และ Recursive Case สรุปได้ว่า

$$\text{ways}(n) = \begin{cases} 1, & \text{ถ้า } n = 0 \\ 0, & \text{ถ้า } n < 0 \\ \text{ways}(n-1) + \text{ways}(n-3) + \text{ways}(n-5), & \text{ถ้า } n > 0 \end{cases}$$

Recursive Solution

เป็นการเอา Recurrence Relation มาเขียนโค้ดตรง ๆ จะได้

```
int solve(int n) {
    if (n < 0) return 0;
    if (n == 0) return 1;
    return solve(n-1) + solve(n-3) + solve(n-5);
}
```

เวลาการทำงานเป็น $O(3^n)$ แต่ละครั้งเราเรียก 3 ปัญหาย่อย

Top-down Solution

เพิ่มส่วน memo เพื่อที่จะได้ไม่ต้องคำนวณซ้ำในกรณีที่คำนวณไปแล้ว และยอมคำนวณในกรณีที่ยังไม่ได้คำนวณและอย่าลืมเก็บค่าที่คำนวณแล้วด้วย

```
int memo[MAXN];
int frog(int n) {
    if (n < 0) return 0;
    if (n == 0) return 1;
    if (memo[n] != 0) return memo[n];
    return memo[n] = frog(n - 1) + frog(n - 3) + frog(n - 5);
}
```

เวลาการทำงานเป็น $O(n)$ เพราะเราคำนวณ frog(k) แค่ครั้งเดียวสำหรับ k ตั้งแต่ 0 ถึง n

Bottom-up Solution

เราก็เติม Base Case ก่อนที่จะวนรอบ ในกรณีนี้เราจะวนลูปได้เมื่อ $dp[i-1], dp[i-3]$ และ $dp[i-5]$ สามารถคำนวณได้ ดังนั้นเราต้องเติมจน $i-5$ ไม่ติดลบ เพราะจะทำให้ index ของตารางติดลบ เราจึงเติม Base Case ให้ถึง 4 เพื่อให้เริ่มวนลูปที่ 5 ได้

```
int dp[MAXN];
int frog(int n) {
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 1;
    dp[3] = 2;
    dp[4] = 3;
    for(int i=5; i<=n; i++)
        dp[i] = dp[i-1] + dp[i-3] + dp[i-5];
    return dp[n];
}
```

หากเราไม่อยากคำนวณ Base Cases ด้วยมือ ตั้งแต่ dp[1] ถึง dp[4] ก็เขียนเงื่อนไขดักไว้ไม่ให้เกิดเหตุการณ์เรียก index ที่ติดลบ

```
int solve(int n) {
    vector<long long> dp(n + 1, 0);
    dp[0] = 1; // Base Case
    for (int i = 1; i <= n; ++i) {
        if (i >= 1) dp[i] += dp[i - 1];
        if (i >= 3) dp[i] += dp[i - 3];
        if (i >= 5) dp[i] += dp[i - 5];
    }
    return dp[n];
}
```

ซึ่งทั้งสองแบบเวลาการทำงานเป็น $O(n)$

จริงๆ ตัวอย่างโจทย์นี้ คล้ายกับข้อก่อนหน้า เพียงแต่ต้องการแสดงให้เห็นว่าในบางปัญหาสามารถมีมากกว่าสองตัวเลือกได้

2.3 Coin Change: Minimum Coins

สมมติว่าเรากำลังยืนอยู่หน้าเครื่องขายของอัตโนมัติ เราต้องการซื้อขนมที่ราคา 37 บาท เราหยอดเงิน 100 บาทเข้าไป เครื่องจะต้องทอนเงินคืนให้ 63 บาท สมมติว่าเครื่องมีเหรียญไม่จำกัดจำนวนในชนิดต่างๆ: 1 บาท, 5 บาท, 10 บาท และ 50 บาท คำถามคือ เครื่องควรเลือกใช้เหรียญชนิดใดและจำนวนเท่าใด เพื่อทอนเงินคืนคุณด้วยจำนวนเหรียญที่น้อยที่สุด

โดยทั่วไปคนส่วนใหญ่จะตอบทันทีว่า "ก็เอาเหรียญที่ใหญ่ที่สุดมาใช้ก่อน" หรือที่เรียกว่า "ขั้นตอนวิธีเชิงละโมบ" (Greedy Algorithm) สำหรับกรณีนี้ขั้นตอนวิธีเชิงละโมบใช้ได้ โดยเราเลือกเหรียญ 50 บาท 1 เหรียญ (เหลือ 13 บาท), ตามด้วยเหรียญ 10 บาท 1 เหรียญ (เหลือ 3 บาท), และสุดท้ายคือเหรียญ 1 บาท 3 เหรียญ รวมเป็น 5 เหรียญ ซึ่งก็คือจำนวนเหรียญที่น้อยที่สุดจริงๆ

แต่ถ้าเป็นชุดเหรียญอื่นจะได้ผลเช่นเดียวกันไหม ลองพิจารณากรณีต่อไปนี้ เครื่องมีแค่เหรียญ 1 บาท, 3 บาท และ 4 บาท และต้องการทอนเงิน 6 บาท หากใช้ขั้นตอนวิธีเชิงละโมบ ก็จะได้ เลือกเหรียญ 4 บาท (เหลือ 2 บาท), ตามด้วยเลือกเหรียญ 1 บาท (เหลือ 1 บาท) และเลือกเหรียญ 1 บาท (เหลือ 0 บาท) รวมเป็น 3 เหรียญ (4, 1, 1) แต่คำตอบที่ถูกต้องคือ 2 เหรียญ (3, 3)

ทีนี้เราจะทำอย่างไรดี การลองทุกชุดคำตอบที่เป็นไปได้ก็ดูเหมือนจะใช้เวลานานเกินไป เราจะมาลองแก้ด้วย DP กัน

2.3.1 รูปแบบเป็นทางการของปัญหา

โจทย์ข้อที่ 4: Coin Change: Minimum Coins

คุณมีเหรียญอย่างไม่จำกัดจำนวนใน k ชนิด โดยมีมูลค่า $c_1, c_2, ..., c_k$ ตามลำดับ จงหาจำนวนเหรียญที่น้อยที่สุดที่ใช้ในการทอนเงินให้ได้จำนวนเงิน n บาท หากไม่สามารถทอนเงินได้ ให้แสดงผลลัพธ์เป็น -1

ข้อมูลนำเข้า

บรรทัดแรก: จำนวนเต็มบวก k และ n $(1 \le k \le 10, 1 \le n \le 10,000)$

บรรทัดที่สอง: จำนวนเต็มบวก k ตัว แทนมูลค่าของเหรียญแต่ละชนิด $c_1, c_2, ..., c_k$

ข้อมูลส่งออก

จำนวนเต็มเพียงตัวเดียว แทนจำนวนเหรียญที่น้อยที่สุดที่ใช้ทอนเงิน n บาท หรือ -1 ถ้าทอนไม่ได้

ตัวอย่าง

ข้อมูลนำเข้า	ข้อมูลส่งออก
3 6 1 3 4	2
2 7 2 4	-1

คำอธิบายตัวอย่าง:

สำหรับตัวอย่างแรก, $k = 3, n = 6, coins = [1, 3, 4]$

วิธีทอนเงินที่เป็นไปได้:

- 1+1+1+1+1+1 (ใช้ 6 เหรียญ)
- 1+1+4 (ใช้ 3 เหรียญ)
- 3+3 (ใช้ 2 เหรียญ)
- 4+1+1 (ใช้ 3 เหรียญ)

จำนวนเหรียญที่น้อยที่สุดคือ 2 จึงแสดงผลลัพธ์เป็น 2

ลองคิดดูก่อนเปิดหน้าถัดไป

2.3.2 วิเคราะห์ว่าสามารถใช้ DP ในการแก้

ในการประยุกต์กับปัญหา Coin Change: เพื่อหาจำนวนเหรียญที่น้อยที่สุดสำหรับ amount = 6 สมมติว่ามีเหรียญ [1, 3, 5] เราต้องการคำตอบของปัญหาย่อยก่อนหน้านี้ ได้แก่

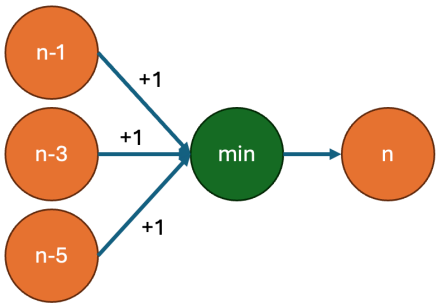
- จำนวนเหรียญที่น้อยที่สุดสำหรับ amount = 5 แล้ว + เหรียญ 1
- จำนวนเหรียญที่น้อยที่สุดสำหรับ amount = 3 แล้ว + เหรียญ 3
- จำนวนเหรียญที่น้อยที่สุดสำหรับ amount = 1 แล้ว + เหรียญ 5

นั่นคือถ้าเรารู้คำตอบที่ดีที่สุดของ amount = 5, amount = 3 และ amount = 1 เราก็สามารถหาคำตอบที่ดีที่สุดของ amount = 6 ได้ ดังนั้น ทีนี้ถ้าเราใช้การวาดรูปในการหาความสัมพันธ์ การที่จะได้คำตอบว่าจำนวนเหรียญน้อยที่สุดเพื่อแลกเป็นเงิน n บาทนั้น มีเหตุการณ์อะไรเกิดได้บ้าง ให้เราพิจารณาสิ่งที่มีอยู่นั่นคือจำนวนเหรียญ $1, 3, 5$ บาท นั่นคือถ้า step ที่จะทำให้เกิดคำตอบที่ n บาท

- ถ้าเราใช้เหรียญ 1 บาท นั่นคือเราต้องการคำตอบ จำนวนเหรียญที่น้อยที่สุดของเงิน $n - 1$ บาท
- ถ้าเราใช้เหรียญ 3 บาท นั่นคือเราต้องการคำตอบ จำนวนเหรียญที่น้อยที่สุดของเงิน $n - 3$ บาท
- ถ้าเราใช้เหรียญ 5 บาท นั่นคือเราต้องการคำตอบ จำนวนเหรียญที่น้อยที่สุดของเงิน $n - 5$ บาท

นั่นคือการที่จะได้คำตอบของวงกลม n บาท เราต้องการวงกลม $n - 1$, $n - 3$ และ $n - 5$ และมีลูกศรชี้มาที่วงกลม “การดำเนินการ” โดยในจุดนี้ เราจะไม่ใช้การ “รวม” ค่าผลลัพธ์ แต่เราจะเลือกผลลัพธ์ที่ “น้อยที่สุด” ซึ่งก็คือการหาค่า min นั่นเอง

สิ่งสำคัญที่ต้องจำไว้ก็คือ หลังจากเลือกผลลัพธ์จาก $n - 1$, $n - 3$ หรือ $n - 5$ แล้ว เราต้องบวกเพิ่มอีก 1 เหรียญเข้าไปเสมอ เพราะถือว่าเราเลือกใช้เหรียญนั้น ๆ ไป 1 เหรียญแล้วในการแลก ดังรูป 2.3



ภาพที่ 2.3: คำตอบของจำนวนเหรียญ n บาทเกิดจากการค่าที่น้อยที่สุดของคำตอบของจำนวนเหรียญ $n - 1$, $n - 3$ หรือ $n - 5$ บวกอีก 1 เหรียญ

สำหรับ Overlapping Subproblems เราพบว่าต้องคำนวณค่าของปัญหาย่อยเดิมๆ ซ้ำหลายครั้ง สมมติว่าเรียกด้วยฟังก์ชัน coinChange

- ทอนเงิน(6) จะเรียก ทอนเงิน(5) (ใช้เหรียญ 1) ทอนเงิน(3) (ใช้เหรียญ 3) และ ทอนเงิน(1) (ใช้เหรียญ 5)
- ทอนเงิน(5) จะเรียก ทอนเงิน(4) (ใช้เหรียญ 1) ทอนเงิน(2) (ถ้าใช้เหรียญ 3) และ ทอนเงิน(0) (ใช้เหรียญ 5)
- ทอนเงิน(4) จะเรียก ทอนเงิน(3) (ใช้เหรียญ 1) ทอนเงิน(1) (ถ้าใช้เหรียญ 3) และ ทอนเงิน(-1) (ใช้เหรียญ 5)
- สังเกตว่า ทอนเงิน(3) ถูกเรียกคำนวณซ้ำหลายครั้งจากเส้นทางที่แตกต่างกัน

2.3.3 ขั้นตอนการแก้ปัญหาแบบ DP

การสร้าง State

สิ่งที่เปลี่ยนแปลงไปในแต่ละปัญหาย่อยคือ amount ที่เราต้องการทอน

ดังนั้น เราจะนิยาม State ได้ดังนี้: coinChange(i) หมายถึง จำนวนเหรียญที่น้อยที่สุดที่ต้องใช้ในการทอนเงินจำนวน i

Recurrence Relation

เพื่อคำนวณ coinChange(i) เราจะลองใช้ทุกเหรียญ c ในชุด coins ที่มีค่าไม่เกิน i นี่เป็นตัวอย่างที่แตกต่างจากโจทย์ก่อนหน้าคือเราต้องลองกับเหรียญทุกแบบ

ข้อสังเกตถ้าเราเลือกใช้เหรียญ c จำนวนเงินที่เหลือจะเป็น $i - c$ และ**เราจะใช้เหรียญ 1 เหรียญ** ดังนั้นจำนวนเหรียญทั้งหมดจะเป็น 1 (สำหรับเหรียญ c ที่เลือก) + coinChange$(i - c)$ (จำนวนเหรียญที่น้อยที่สุดสำหรับยอดที่เหลือ) เนื่องจากเราต้องการ จำนวนเหรียญที่น้อยที่สุด เราจึงต้องเลือกค่าที่น้อยที่สุดจากทุกตัวเลือกของเหรียญ c จะได้ว่า

$$\text{coinChange}(n) = 1 + \min_{c \in \text{coins}} \text{coinChange}(n - c)$$

ส่วน Base Case (กรณีฐาน):

- coinChange$(0) = 0$ (ถ้าต้องการทอนเงิน 0 บาท เราไม่ต้องใช้เหรียญเลย)
- coinChange$(i) = \infty$ (ค่าอนันต์) สำหรับ i < 0 (ไม่สามารถทอนเงินติดลบได้ เลยกำหนดค่าสูง ๆ ไว้ให้จะได้ไม่เป็นคำตอบ)

ดังนั้น สูตร Recurrence Relation คือ:

$$\text{coinChange}(n) = \begin{cases} 0, & \text{ถ้า } n = 0 \\ \infty, & \text{ถ้า } n < 0 \\ 1 + \min_{c \in \text{coins}} \text{coinChange}(n - c), & \text{ถ้า } n > 0 \end{cases}$$

Recursive Solution

เมื่อเราแปลง Recurrence Relation เป็นโค้ด จุดสำคัญที่ควรสังเกตคือสัญลักษณ์ $\min_{c \in \text{coins}}$ ซึ่งหมายถึง “ให้เลือกค่าที่น้อยที่สุดเมื่อเราลองใช้เหรียญแต่ละแบบในชุด coins” เราจึงต้องใช้การวนลูป (loop) เพื่อทดลองใช้เหรียญทุกเหรียญใน coins ในแต่ละรอบเราจะเรียกฟังก์ชันแบบ recursive สำหรับจำนวนเงินที่เหลืออยู่ (amount - coin) แล้วบวกเพิ่ม 1 เข้าไป (เพราะเราใช้เหรียญนั้น 1 เหรียญ) สุดท้าย เราจะเก็บค่าที่น้อยที่สุดที่หาได้จากทุกเหรียญนั้นมาเป็นคำตอบ

```
int coinChangeRecursive(const vector<int>& coins, int amount) {
    // Base case: If amount is 0, no coins are needed
    if (amount == 0) {
        return 0;
    }
    // Base case: If amount becomes negative, this path is not possible
    if (amount < 0) {
        return INT_MAX;
    }
    int minCoins = INT_MAX;
    // Try using each coin in the set
    for (int coin : coins) {
        // Recursively solve the subproblem for (amount - coin)
        int result = coinChangeRecursive(coins, amount - coin);
        // If a valid answer is found (not INT_MAX)
        if (result != INT_MAX) {
            // Update minCoins
            minCoins = min(minCoins, 1 + result);
        }
    }
    return minCoins;
}
```

เวลาการทำงานเป็น $O(k^n)$ เพราะมีการคำนวณซ้ำหลายครั้ง

Top-down Solution

เพื่อให้การคำนวณของเราเร็วขึ้น เราเพิ่ม "memoization" เข้าไปใน Recursive Solution ถ้าเราเคยคำนวณจำนวนเหรียญที่น้อยที่สุดสำหรับ amount ใด ๆ แล้ว เราจะเก็บผลลัพธ์นั้นไว้ในตาราง memo และหากต้องคำนวณค่าซ้ำ เราจะนำค่าที่เคยได้มาใช้งานทันทีโดยไม่ต้องคำนวณซ้ำอีก นอกเหนือจากการเพิ่ม memoization แล้ว โครงสร้างยังเหมือนกับวิธี Recursive เดิมทุกประการ เพียงแค่มีการเช็กผลลัพธ์ในตาราง memo ก่อนเท่านั้น ในส่วนนี้ เราจะประกาศตัวแปร memo เป็นตัวแปร Global Variable จากนั้นเรากำหนดค่าเริ่มต้นของ memo ด้วย memo.assign(n + 1, -1); ใน main เพื่อให้ทุกตำแหน่งเริ่มต้นเป็น -1 แปลว่ายังไม่เคยคำนวณค่าจำนวนเหรียญที่น้อยที่สุดสำหรับค่าจำนวนเงินแต่ละค่า

```
vector<int> memo;
int coinChangeTopDown(const vector<int>& coins, int amount) {
    if (amount == 0) { // Base Cases
        return 0;
    }
    if (amount < 0) { // Base Cases
        return INT_MAX;
    }
    if (memo[amount] != -1) { // Check memo
        return memo[amount];
    }
    // Compute the answer and store the result in the memo
    int minCoins = INT_MAX;
    for (int coin : coins) {
        int result = coinChangeTopDown(coins, amount - coin);
        if (result != INT_MAX) {
            minCoins = min(minCoins, 1 + result);
        }
    }
    memo[amount] = minCoins;
    return minCoins;
}
// Initialize the memo array in main function
memo.assign(n + 1, -1);
```

เวลาการทำงานจะลดลงเป็น $O(n \times k)$

Bottom-up Solution

เริ่มต้นโดยการกำหนดค่า Base Case ลงในตาราง dp หลังจากนั้นเราจะคำนวณหาค่าคำตอบจากจำนวนน้อย ๆ ไปมาก จนถึงจำนวนเงินที่ต้องการ (amount) โดยในแต่ละจำนวนเงิน เราจะลองพิจารณาเหรียญทุกแบบ แล้วเลือกทางเลือกที่ดีที่สุดเก็บไว้ใน dp การที่เราเริ่มจาก Base Case ก่อน ทำให้ในแต่ละขั้น เราสามารถใช้ผลลัพธ์จากกรณีย่อยที่เคยคำนวณไว้แล้ว ช่วยให้สามารถหาคำตอบในขั้นถัดไปได้

```
int coinChangeBottomUp(const vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX); // Create a dp table
    dp[0] = 0; // Base Case
    for (int i = 1; i <= amount; ++i) { // Compute from amount = 1 up to the target
        for (int coin : coins) { // Try every coin
            // If the coin' s value is less than or equal to current amount (i)
            if (i >= coin) {
                // And if the subproblem dp[i - coin] is possible
                if (dp[i - coin] != INT_MAX) { // updatedp[i]
                    dp[i] = min(dp[i], 1 + dp[i - coin]);
                }
            }
        }
    }
    return dp[amount];
}
```

เวลาการทำงานเป็น $O(n \times k)$ จากการวน n รอบแต่ละรอบพิจารณาเหรียญ k เหรียญ

2.4 House Robber

สมมติว่าคุณเป็นโจรมืออาชีพ ภารกิจในคืนนี้คือ ปล้นเงินให้ได้มากที่สุดจากแถวของบ้านในหมู่บ้าน มีข้อห้ามเพียงข้อเดียวคือ ห้ามปล้นบ้านสองหลังที่ติดกัน แนวคิดแรกที่อาจจะคิดได้แบบเร็วๆ คือเลือกบ้านที่มีมูลค่าสูงสุดก่อน แต่นั่นอาจจะไม่ใช่คำตอบที่ดีที่สุดเสมอไป ลองพิจารณามูลค่าของแถวของบ้านในหมู่บ้าน [2, 8, 9, 7, 1]

- **ทางเลือกมูลค่าสูงสุดก่อน** เลือกปล้นบ้าน 9 และบ้านที่เหลือ ได้เงิน 9 + 2 + 1 = 12
- **ทางเลือกที่ได้มูลรวมค่าสูงสุด** ข้ามบ้าน 9 ไปก่อน เลือกปล้นบ้าน 8 และ 7 แทน ได้เงิน 8 + 7 = 15

นั่นคือหากใช้วิธีการเชิงละโมบอาจจะไม่ได้คำตอบที่ดีที่สุดโดยรวม ในหัวข้อย่อยนี้เรามาวิเคราะห์ปัญหานี้กัน

2.4.1 รูปแบบเป็นทางการของปัญหา

โจทย์ข้อที่ 5: House Robber

คุณเป็นโจรมืออาชีพที่วางแผนจะปล้นบ้านในย่านหนึ่ง ซึ่งมีบ้านเรียงเป็นแถวจำนวน n หลัง แต่ละบ้านมีเงินจำนวนหนึ่งซ่อนอยู่ ปัญหาสำคัญคือ ถ้าคุณปล้นบ้านสองหลังที่อยู่ติดกันในคืนเดียวกัน ระบบรักษาความปลอดภัยจะทำงานและตำรวจจะมาถึงทันที ดังนั้นคุณต้องการหาวิธีการเลือกบ้านที่จะปล้นเพื่อให้ได้เงินสูงสุดโดยไม่ปล้นบ้านที่ติดกัน จงหาจำนวนเงินสูงสุดที่คุณสามารถปล้นได้ในคืนนั้น

ข้อมูลนำเข้า

บรรทัดแรกประกอบด้วยตัวเลขจำนวนเต็ม n แทนจำนวนบ้าน

บรรทัดที่สองประกอบด้วยตัวเลขจำนวนเต็ม n ตัว แทนจำนวนเงินในแต่ละบ้านตั้งแต่บ้านแรกถึงบ้านสุดท้าย

ข้อมูลส่งออก

จำนวนเต็มเพียงตัวเดียว แทนจำนวนเงินสูงสุดที่สามารถปล้นได้

ตัวอย่าง

ข้อมูลนำเข้า	ข้อมูลส่งออก
4 1 2 3 1	4
5 2 7 9 3 1	12

คำอธิบายตัวอย่าง:

สำหรับตัวอย่างที่สอง (n=5, nums=[2, 7, 9, 3, 1]) วิธีการปล้นที่ให้เงินสูงสุดคือ 12 หน่วย ซึ่งเกิดจากลำดับการเลือกบ้านดังนี้:

- เลือกปล้นบ้านหลังที่ 0 (ได้เงิน 2), บ้านหลังที่ 2 (ได้เงิน 9) และบ้านหลังที่ 4 (ได้เงิน 1) รวมเป็นเงิน 2 + 9 + 1 = 12 หน่วย

ทางเลือกอื่น เช่น การปล้นบ้านหลังที่ 1 (ได้ 7) และบ้านหลังที่ 3 (ได้ 3) จะได้เงินเพียง 10 หน่วย

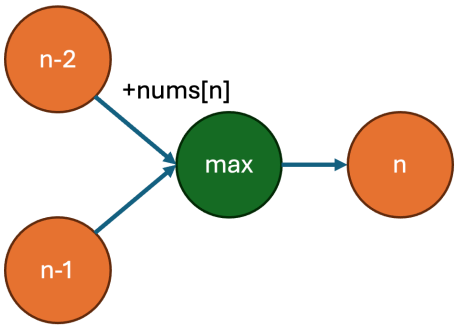
ลองคิดดูก่อนเปิดหน้าถัดไป

2.4.2 วิเคราะห์ว่าสามารถใช้ DP ในการแก้

ลองพิจารณาการตัดสินใจเมื่อเราอยู่ที่บ้านหลังสุดท้าย (วงกลม n) เรามีทางเลือกเพียง 2 ทางเท่านั้น คือ

- **ปล้นบ้านหลังที่** n คิดดูว่าจะเกิดเหตุการณ์อะไรบ้าง เราจะได้เงินของบ้านหลังที่ n nums[n] (ตรงนี้ทำให้เราต้องเขียนเลขเงินของบ้านที่ n ไปบนเส้นด้วย) และเรา **ห้าม** ปล้นบ้านหลังที่ $n-1$ ดังนั้น เราต้องไปถามคำตอบจากการปล้นบ้านหลังที่ 0 ถึง $n-2$ จะได้ว่ามีเส้นจากวงกลม $n-2$ ที่มีเลข nums[n] อยู่บนเส้น ไปที่การดำเนินการ max
- **ไม่ปล้นบ้านหลังที่** n คิดดูว่าจะเกิดเหตุการณ์อะไรบ้าง ก็แสดงเราต้องไปถามคำตอบจากการปล้นบ้านหลังที่ 0 ถึง $n-1$ จะได้ว่ามีเส้นจากวงกลม $n-1$ ไปที่การดำเนินการ max

เราต้องเลือกค่าที่มากกว่าระหว่าง 2 ทางเลือกนี้ จะได้ว่า max(ทางเลือกที่ 1, ทางเลือกที่ 2) ดังภาพที่ 2.4



ภาพที่ 2.4: คำตอบของจำนวนเงินมากที่สุดจากบ้านหลังแรกถึงบ้านหลังที่ n เกิดจากการเลือกค่ามากสุดของคำตอบเลือกบ้านหลังที่ n รวมกับคำตอบของของจำนวนเงินมากที่สุดจากบ้านหลังแรกถึงบ้านหลังที่ $n-2$ กับ คำตอบของจำนวนเงินมากที่สุดจากบ้านหลังแรกถึงบ้านหลังที่ $n-1$

ในส่วน Optimal Substructure ลองคิดตัวอย่าง เพื่อหาคำตอบสำหรับบ้านหลังแรกถึงหลังที่ 6 เราต้องการคำตอบของบ้านหลังแรกถึงหลังที่ 5 และ 4 เช่นเดียวกันเพื่อหาคำตอบสำหรับบ้านหลังแรกถึงหลังที่ 5 เราต้องการคำตอบของบ้าน 4 และ 3 เห็นได้ว่าเราต้องคำนวณหาคำตอบของบ้านหลังแรกถึงหลังที่ 4 ซ้ำกันแล้วถ้าปัญหาใหญ่ขึ้น ปัญหาย่อยเหล่านี้จะถูกคำนวณซ้ำซ้อนกันเป็นจำนวนมาก

2.4.3 ขั้นตอนการแก้ปัญหาแบบ DP

การสร้าง State

ในปัญหานี้ “เรากำลังพิจารณาบ้านถึงหลังที่เท่าไหร่” เราจะกำหนด State ได้ดังนี้: dp(i) = จำนวนเงินสูงสุดที่สามารถปล้นได้จากบ้านหลังแรก (index 0) จนถึงบ้านหลังที่ i

Recurrence Relation

เราก็อ้างอิงจากการวิเคราะห์ Optimal Substructure ที่เราทำไว้จะได้ว่าในกรณี Recursive Case มี 2 ทางเลือก คือ

- ทางเลือกที่ 1 (ปล้นบ้าน n): nums[n] + dp($n-2$)
- ทางเลือกที่ 2 (ไม่ปล้นบ้าน n): dp($n-1$)

ซึ่งเราเลือกทางเลือกที่ได้ค่ามากที่สุด ดังนั้น Recurrence Relation คือ: $dp[n] = max(dp[n-1], nums[n]+dp[n-2])$
สำหรับการกำหนด Base Case นั้น เราแบ่งได้ 2 กรณี

- dp[0]: ถ้ามีแค่บ้านเดียว (บ้านหลังที่ 0) ก็มีทางเลือกเดียวคือเลือกบ้านนั้นเลย ดังนั้น dp[0] = nums[0]
- dp[1]: ถ้ามี 2 บ้าน เราต้องเลือกบ้านที่มีเงินมากกว่า ดังนั้น dp[1] = max(nums[0], nums[1])

เมื่อรวมกันจะได้

$$dp(n) = \begin{cases} \text{nums}[0], & \text{ถ้า } n = 0 \\ \max(\text{nums}[0], \text{nums}[1]), & \text{ถ้า } n = 1 \\ \max(dp(n-1), \text{nums}[n] + dp(n-2)), & \text{ถ้า } n \geq 2 \end{cases}$$

หรืออาจจะปรับ Base Cases เป็นกรณีเดียวคือ $n < 0$ ให้มีค่าเป็น 0 ก็ได้เช่นกัน

$$dp(n) = \begin{cases} 0, & \text{ถ้า } n < 0 \\ \max(dp(n-1), \text{nums}[n] + dp(n-2)), & \text{ถ้า } n \geq 0 \end{cases}$$

Recursive Solution

นี่คือการแปลง Recurrence Relation ไปเป็นโค้ดตรงๆ

```
vector<int> nums;
int robRecursive(int i) {
    // Base case: no house
    if (i < 0) {
        return 0;
    }
    // Base case: one house
    if (i == 0) {
        return nums[0];
    }
    // Recursive case
    return max(robRecursive(i - 1), nums[i] + robRecursive(i - 2));
}
```

เวลาในการทำงานเป็น $O(2^n)$ ที่การเรียกฟังก์ชันแต่ละครั้ง เราเรียกตัวเองอีก 2 ครั้ง ($i - 1$และ $i - 2$) ทำให้เกิดต้นไม้การเรียกซ้ำ (Recursion Tree) ที่มีขนาดใหญ่ขึ้นแบบทวีคูณ ช้ามากสำหรับข้อมูลเข้าขนาดใหญ่

Top-down Solution

ปรับปรุงจาก Recursive Solution โดยการ “จดจำ” ผลลัพธ์ของปัญหาย่อยที่เคยคำนวณไว้แล้ว เพื่อไม่ต้องคำนวณซ้ำ

```
vector<int> nums;
vector<int> memo;
int rob_topdown(int i) {
    // init memo
    if (memo.empty()) {
        memo.assign(nums.size(), -1);
    }
    // Base cases
    if (i < 0) return 0;
    if (i == 0) return nums[0];
    // If already computed, return the stored value.
    if (memo[i] != -1) {
        return memo[i];
    }
    // if it hasn't been calculated yet, calculate it
    int result = max(rob_topdown(i - 1), nums[i] + rob_topdown(i - 2));
    // Store the result in memo before returning it
    memo[i] = result;
    return result;
}
```

เวลาในการทำงานเป็น $O(n)$ เรามี State ทั้งหมด n ตัว (dp[0] ถึง dp[$n - 1$]) ด้วยการใช้ memo แต่ละ state จะถูกคำนวณเพียงครั้งเดียวเท่านั้น ดังนั้นเวลาทั้งหมดจึงขึ้นอยู่กับจำนวน state

Bottom-up Solution

สร้างตารางขึ้นมาและเริ่มเติมค่าจาก Base Case ไปจนถึงปัญหาที่ใหญ่ที่สุด

```
vector<int> nums;
int rob_bottom_up() {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];
    vector<int> dp(n);
    // Base Cases
    dp[0] = nums[0];
    dp[1] = max(nums[0], nums[1]);
    // fill the table from the 2nd house
    for (int i = 2; i < n; ++i) {
        dp[i] = max(dp[i - 1], nums[i] + dp[i - 2]);
    }
    return dp[n - 1];
}
```

เวลาในการทำงานเป็น $O(n)$ เราใช้ loop เดียวในการเติมตาราง dp ซึ่งวน $n-2$ รอบคิดเป็น $O(n)$

จาก Recurrence Relation dp[i] = max(dp[$i-1$], nums[i] + dp[$i-2$]) จะเห็นว่าการคำนวณค่า dp[i] นั้น ขึ้นอยู่กับแค่ค่า 2 ตัวก่อนหน้า (dp[$i-1$] และ dp[$i-2$]) เท่านั้น เราไม่จำเป็นต้องเก็บตาราง dp ทั้งหมด เราสามารถใช้ตัวแปรแค่ 2 - 3 ตัวในการเก็บค่าเหล่านี้ได้

```
vector<int> nums;
int rob_optimized() {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];
    // prev2 represents dp[i-2], and prev1 represents dp[i-1]
    int prev2 = nums[0];
    int prev1 = max(nums[0], nums[1]);
    for (int i = 2; i < n; ++i) {
        int current = max(prev1, nums[i] + prev2);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

2.5 Maximum Subarray Sum

2.5.1 รูปแบบเป็นทางการของปัญหา

โจทย์ข้อที่ 6: The Maximum Subarray Sum Problem

ให้อาร์เรย์ของจำนวนเต็ม arr ขนาด n จงหาผลรวมที่มากที่สุดของ subarray ที่ไม่ว่างเปล่า (non-empty subarray) ที่มีสมาชิกติดกัน

ข้อมูลนำเข้า

บรรทัดแรกมีจำนวนเต็ม n ($1 \leq n \leq 10^5$) แทนขนาดของอาร์เรย์

บรรทัดที่สองมีจำนวนเต็ม n จำนวน ($-10^4 \leq arr[i] \leq 10^4$) แทนสมาชิกของอาร์เรย์

ข้อมูลส่งออก

จำนวนเต็มหนึ่งตัว แทนผลรวมสูงสุดของ subarray ที่เป็นไปได้

ตัวอย่าง

ข้อมูลนำเข้า	ข้อมูลส่งออก
9 -2 1 -3 4 -1 2 1 -5 4	6
3 -5 -1 -3	-1

คำอธิบายตัวอย่าง:

สำหรับตัวอย่างแรก, อาร์เรย์ที่ได้รับคือ [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Subarray ที่ให้ผลรวมสูงสุดคือ [4, -1, 2, 1]

ซึ่งมีผลรวมเท่ากับ 4 + (-1) + 2 + 1 = 6

ดังนั้น คำตอบที่ต้องส่งออกคือ 6

สำหรับตัวอย่างที่สอง, อาร์เรย์ประกอบด้วยจำนวนลบทั้งหมด [-5, -1, -3]

Subarray ที่ให้ผลรวมสูงสุดคือ [-1] ซึ่งเป็น subarray ที่มีขนาดเล็กที่สุดและมีค่ามากที่สุด

ดังนั้น คำตอบที่ต้องส่งออกคือ -1

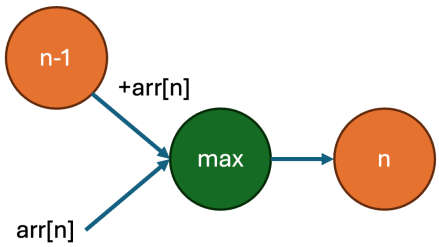
ลองคิดดูก่อนเปิดหน้าถัดไป

2.5.2 วิเคราะห์ว่าสามารถใช้ DP ในการแก้

สมมุติว่าพิจารณาอาร์เรย์แต่ละตัวมาจนถึงตำแหน่งที่ n เราจะมีสองทางเลือก

- **นับรวมต่อ** กรณีนี้คือการเอาผลรวมสูงสุดที่สิ้นสุดที่ตำแหน่ง $n-1$ รวมกับค่าของ $arr[n]$ เข้ามาด้วย เราก็เขียนวงกลม $n-1$ (แทนผลรวมสูงสุดที่สิ้นสุดที่ตำแหน่ง $n-1$) แล้วลากลูกศรมาที่วงกลมการดำเนินการ ทั้งนี้เขียนบวก $arr[n]$ ตรงลูกศรด้วย
- **เริ่มนับใหม่** เริ่มนับ subarray ใหม่ที่ n กรณีนี้คือในบางครั้งถ้าผลรวมก่อนหน้าเป็นค่าลบ การเริ่มต้นนับใหม่ที่ n (ใช้แค่ $arr[n]$) จะให้ผลดีกว่า เราก็ไม่วาดวงกลม แต่เขียนค่า $arr[n]$ แล้วลากเส้นมาที่การดำเนินการ

แล้วการดำเนินการของข้อนี้คืออะไร max นั่นเอง จะทำให้ได้ภาพที่ 2.5



ภาพที่ 2.5: คำตอบของผลรวมสูงสุดที่สิ้นสุดที่ตำแหน่ง n เกิดจากการเลือกค่ามากสุดระหว่างการรวม arr[n] เข้ากับผลรวมสูงสุดที่สิ้นสุดที่ตำแหน่ง $n-1$ กับค่า arr[n]

สำหรับ Overlapping Subproblems เมื่อเราต้องการหาผลรวมสูงสุดของ subarray ที่ลงท้ายที่ตำแหน่ง i เราจะต้องใช้ผลรวมสูงสุดของ subarray ที่ลงท้ายที่ตำแหน่ง $i-1$ มาคำนวณด้วย และเมื่อเราจะหาค่าของตำแหน่ง $i+1$ ต่อไป เราก็ต้องรู้ผลรวมสูงสุดของ subarray ที่ลงท้ายที่ i อีก การที่เราต้องเรียกใช้ค่าของ subarray ที่ลงท้ายแต่ละตำแหน่งซ้ำ ๆ อยู่ตลอดแสดงถึงปัญหาย่อยที่ซ้ำกัน

2.5.3 ขั้นตอนการแก้ปัญหาแบบ DP

การสร้าง State

เราจะนิยาม State ได้ดีที่สุดดังนี้:

$$dp[i] \text{ คือ ผลรวมสูงสุดของ subarray ที่ลงท้ายด้วยสมาชิกตำแหน่งที่ } i$$

คำเตือน dp[i] ไม่ใช่ ผลรวมสูงสุดของ subarray ในช่วง [0...i] แต่เป็นผลรวมสูงสุดของ subarray ที่ จบตรงตำแหน่ง i เท่านั้น

Recurrence Relation

จาก Optimal Substructures จะได้ว่า dp(i) = max(arr[i], dp($i-1$) + arr[i]) จากนั้นเรามาคิดต่อในส่วนของ Base Case ซึ่งจะพบว่าหาก array มีตัวเดียว อย่างไรก็ต้องเลือก arr[0] มาเป็นคำตอบ ทำให้ได้ว่า

$$dp(n) = \begin{cases} arr[0] & \text{if } n = 0, \\ \max(arr[n], dp(n-1) + arr[n]) & \text{if } n > 0. \end{cases}$$

และคำตอบสุดท้ายคือ

$$\text{answer} = \max_{0 \le i < n} dp(i)$$

Recursive Solution

เป็นการแปลง Recurrence Relation มาเป็นโค้ดโดยตรง แต่ว่ามีส่วนคำตอบสุดท้ายที่ต้องวนหาค่าดูได้ในส่วน main function

```
#include <bits/stdc++.h>
using namespace std;

int maxSubArrayRecursive(const vector<int>& arr, int i) {
    // Base case
    if (i == 0) {
        return arr[0];
    }
    // Recurrence relation
    return max(arr[i], maxSubArrayRecursive(arr, i - 1) + arr[i]);
}
int main() {
    vector<int> arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n = arr.size();
    if (n == 0) {
        return 0;
    }
    int overall_max = INT_MIN;
    // call the function for every position to find the overall maximum value
    for (int i = 0; i < n; ++i) {
        overall_max = max(overall_max, maxSubArrayRecursive(arr, i));
    }
    cout << overall_max << endl;
    return 0;
}
```

เวลาการทำงานเป็น $O(n^2)$

Top-down Solution

เพิ่ม memo เพื่อที่จะได้ไม่ต้องคำนวณส่วนที่เคยคำนวณมาแล้ว ในส่วนของ main function นั้นจะมีการกำหนดขนาดและค่าเริ่มต้นของ memo

```
#include <bits/stdc++.h>
using namespace std;
vector<int> memo;
int maxSubArrayTopDown(const vector<int>& arr, int i) {
    // Base case
    if (i == 0) {
        return arr[0];
    }
    // Check memo
    if (memo[i] != -1) {
        return memo[i];
    }
    // Compute the value and store it in memo
    memo[i] = max(arr[i], maxSubArrayTopDown(arr, i - 1) + arr[i]);
    return memo[i];
}
int main() {
    vector<int> arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n = arr.size();
    if (n == 0) {
        return 0;
    }
    // Set the size and initial value of the memo
    memo.resize(n, -1);
    int overall_max = INT_MIN;
    for (int i = 0; i < n; ++i) {
```

```
        overall_max = max(overall_max, maxSubArrayTopDown(arr, i));
    }
    cout << overall_max << endl;
    return 0;
}
```

เวลาการทำงานเป็น $O(n)$

Bottom-up Solution

ในส่วนนี้จะทำการกำหนดค่า Base Case ให้กับตาราง dp ก่อนหลังจากนั้นจึงวนลูปถัดจาก Base Case ไปจนถึงจำนวน n ที่ต้องการ ทั้งนี้เราจะหาค่า overall_max ไปในลูปด้วยเลย

```
#include <bits/stdc++.h>
using namespace std;
int maxSubArrayBottomUp(const vector<int>& arr) {
    int n = arr.size();
    if (n == 0) {
        return 0;
    }
    vector<int> dp(n);
    // Base case
    dp[0] = arr[0];
    int overall_max = dp[0];
    // Fill the dp table from left to right
    for (int i = 1; i < n; ++i) {
        // Recurrence relation
        dp[i] = max(arr[i], dp[i - 1] + arr[i]);
        // Update the overall maximum
        overall_max = max(overall_max, dp[i]);
    }
    return overall_max;
}
int main() {
    vector<int> arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int result = maxSubArrayBottomUp(arr);
    cout << result << endl;
    return 0;
}
```

เวลาการทำงานเป็น $O(n)$ จากสมการ dp[i] = max(arr[i], dp[$i-1$] + arr[i]) เราจะเห็นว่าการคำนวณ dp[i] นั้นขึ้นอยู่กับเพียง dp[$i-1$] เท่านั้น เราไม่จำเป็นต้องเก็บตาราง dp ทั้งหมด เราสามารถใช้ตัวแปรแค่ 2 ตัวคือ current_max (แทน dp[i]) และ global_max (แทนคำตอบสุดท้าย) ได้ วิธีนี้เรียกว่า **Kadane's Algorithm**

```
#include <bits/stdc++.h>
using namespace std;
// This method is called Kadane's Algorithm
int maxSubArrayOptimized(const vector<int>& arr) {
    int n = arr.size();
    if (n == 0) {
        return 0;
    }
    int current_max = arr[0];
    int global_max = arr[0];
    for (int i = 1; i < n; ++i) {
        // Update current_max
        current_max = max(arr[i], current_max + arr[i]);
        // Update global_max
        global_max = max(global_max, current_max);
    }
    return global_max;
}
```

```
int main() {
    vector<int> arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int result = maxSubArrayOptimized(arr);
    cout << result << endl;
    return 0;
}
```

2.6 Prefix Sum

หนึ่งในปัญหาที่เราพบได้บ่อยที่สุด (อย่างน้อยก็พบใน TOI จนแทบเป็นเครื่องมือที่ใช้เป็นเรื่องปกติ) ไม่ว่าจะเป็นปัญหาเกี่ยวกับสถิติ การประมวลผลสัญญาณ หรือแม้แต่การวิเคราะห์ข้อมูลง่าย ๆ ก็คือ การต้องหาผลรวมของข้อมูลในช่วงใดช่วงหนึ่งของอาร์เรย์เดิม หลาย ๆ ครั้ง

ลองพิจารณากรณีต่อไปนี้ดู สมมติว่าเรามีอาร์เรย์ของราคาหุ้นย้อนหลัง 100,000 วัน และเรามีคำถาม 100,000 ข้อที่ถามว่า "ผลรวมราคาหุ้นจากวันที่ A ถึงวันที่ B เป็นเท่าไหร่?"

วิธีที่ตรงไปตรงมาที่สุดก็คือการวนลูปจากวันที่ A ถึง B ในแต่ละครั้ง แต่ถ้าหากขนาดของอาร์เรย์และจำนวนคำถามมีจำนวนมาก วิธีแบบตรงไปตรงมาจะต้องทำงานถึง 100,000 * 100,000 แค่ 10,000,000,000 ครั้งเอง.. ซึ่งเป็นตัวเลขที่ทำให้โปรแกรมของเรา "Time Limit Exceeded" อย่างแน่นอน

2.6.1 รูปแบบเป็นทางการของปัญหา

โจทย์ข้อที่ 7: Prefix Sum (Quick Sum)

ให้อาร์เรย์ arr ขนาด n และคำถาม q คำถาม แต่ละคำถามจะกำหนดช่วง $[l, r]$ (0-based index) จงหาผลรวมของสมาชิกในอาร์เรย์ตั้งแต่ตำแหน่ง l ถึง r

ข้อมูลนำเข้า

- บรรทัดแรก จำนวนเต็ม n แทนขนาดของอาร์เรย์ ($1 \leq n \leq 10^5$)
- บรรทัดที่สอง จำนวนเต็ม n จำนวน แทนสมาชิกในอาร์เรย์ arr
- บรรทัดที่สาม จำนวนเต็ม q แทนจำนวนคำถาม ($1 \leq q \leq 10^5$)
- ถัดไป q บรรทัด แต่ละบรรทัดมีจำนวนเต็มสองตัว l และ r แทนช่วงที่ต้องการหาผลรวม ($0 \leq l \leq r < n$)

ข้อมูลส่งออก สำหรับแต่ละคำถาม ให้แสดงผลรวมของสมาชิกในช่วง $[l, r]$ บนบรรทัดใหม่

ตัวอย่าง

ข้อมูลนำเข้า	ข้อมูลส่งออก
5 1 2 3 4 5 3 0 4 1 3 2 2	15 9 3

คำอธิบายตัวอย่าง:

- สำหรับคำถามแรก $l = 0, r = 4$ ผลรวมคือ $arr[0] + arr[1] + arr[2] + arr[3] + arr[4] = 1 + 2 + 3 + 4 + 5 = 15$
- สำหรับคำถามที่สอง $l = 1, r = 3$ ผลรวมคือ $arr[1] + arr[2] + arr[3] = 2 + 3 + 4 = 9$
- สำหรับคำถามที่สาม $l = 2, r = 2$ ผลรวมคือ $arr[2] = 3$

ลองคิดดูก่อนเปิดหน้าถัดไป

หากแก้แบบตรงไปตรงมา เราจะพบว่าสำหรับทุกคำถาม เราจะวนลูปจาก l ถึง r เพื่อคำนวณผลรวม ดังนั้นการถาม 1 ครั้ง ใช้เวลา $O(n)$

```
long long sum = 0;
for (int i = l; i <= r; ++i) {
    sum += arr[i];
}
```

เหมือนจะดี แต่วิธีนี้มีข้อเสียคือถ้าจำนวนครั้งในการถาม q มีค่ามาก (เช่น 10^5) และ n ก็มาก (เช่น 10^5) วิธีนี้จะใช้เวลา $O(q \times n)$ ซึ่งช้าเกินไป

ลองมองแบบนี้สมมติว่าเราขับรถรถของเราก็จะมีตัววัดระยะทาง (Odometer) ที่เก็บระยะทางตั้งแต่ต้นทางไว้ สมมติว่าเราขับจากต้นทางผ่านจุด A แล้วไปถึงจุด B ถ้าเราอยากรู้ว่าระยะทางจากจุด A ถึงจุด B เป็นเท่าไหร่ เราไม่จำเป็นต้องขับรถใหม่จากจุด A ไป B สิ่งที่เราทำคือดูว่ารถวิ่งจากต้นทางมาถึงจุด B ทั้งหมดกี่กิโลเมตร แล้วลบด้วยระยะทางตั้งแต่ต้นทางถึงจุด A นั่นคือ

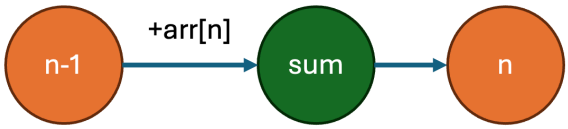
ระยะทาง(A ถึง B) = ระยะทาง(ต้นทาง ถึง B) - ระยะทาง(ต้นทาง ถึง A)

เทคนิค Quicksum นี้แทนที่จะคำนวณผลรวมใหม่ทุกครั้ง เราแค่ต้องคำนวณ “ผลรวมสะสม” (Cumulative Sum) จากต้นอาร์เรย์ไว้ก่อนครั้งเดียว หลังจากนั้น ไม่ว่าจะถามผลรวมในช่วงไหน เราสามารถตอบได้อย่างรวดเร็วด้วยการเอาสองระยะมาลบกัน ทำให้ลดเวลาการทำงานจาก $O(n * q)$ ไปสู่ $O(n + q)$ เมื่อ q เป็นจำนวนครั้งในการถาม อย่างก็ตามเรามาลองวิเคราะห์แบบ DP ในปัญหานี้ดูกัน

2.6.2 วิเคราะห์ว่าสามารถใช้ DP ในการแก้

อย่างแรก Optimal Substructure จริง ปัญหานี้เหมือนกับชีวิตจริงของเรา ลองดูสถานการณ์นี้ สมมติว่าให้เราบวกเลข 2 ตัว เราจะบวกอย่างไร เราก็เอาตัวแรกบวกตัวที่สอง ถ้ามีเลข 3 ตัวหละ เราก็อาจจะบวกเลขตั้งแต่ตัวแรก ตัวที่สอง ตัวที่สาม ถ้ามีมากกว่านั้นหละ เช่น 100 ตัว เราก็คงจะไม่บวกเลขตั้งแต่ตัวที่ 1 เราก็เอาผลรวมของตัวแรกถึงตัวที่ 99 มาใช้ แล้วค่อยมาบวกกับตัวที่ 100

พอพิจารณาเป็นรูป การหาผลรวมถึงตัวที่ n ก็คือ การนำเอาผลรวมถึงตัวที่ $n-1$ รวมกับค่าตัวที่ n ทำให้ได้ดังรูป 2.6



ภาพที่ 2.6: คำตอบของผลรวมถึงตำแหน่ง n เกิดจากผลรวมถึงตำแหน่ง $n-1$ กับค่า arr[n]

สำหรับ Overlapping Subproblems เรามาลองดูผลรวมในช่วงต่าง ๆ

- ผลรวมจากต้นถึง index 4: sum$(0,4)$ = arr[0] + arr[1] + arr[2] + arr[3] + arr[4]
- ผลรวมจากต้นถึง index 5: sum$(0,5)$ = arr[0] + arr[1] + arr[2] + arr[3] + arr[4] + arr[5]

จะเห็นได้ว่า sum$(0,5)$ ต้องการคำนวณผลรวมของ arr[0] ถึง arr[4] ซึ่งก็คือ sum$(0,4)$ นั่นเอง ถ้าเรามีคำถาม sum$(0,4)$ และ sum$(0,5)$ การคำนวณแบบ Naive จะทำการคำนวณส่วนที่ซ้ำกัน (arr[0] ถึง arr[4]) ซ้ำอีกครั้ง นี่คือ Overlapping Subproblems เราสามารถเก็บผลลัพธ์ของ sum$(0,4)$ ไว้แล้วนำมาใช้ใหม่เพื่อคำนวณ sum$(0,5)$ ได้

2.6.3 ขั้นตอนการแก้ปัญหาแบบ DP

การสร้าง State

เรานิยาม State ได้ดังนี้ dp[i] = ผลรวมของสมาชิกในอาร์เรย์ arr จาก index 0 ถึง index i (คือ arr[0] + arr[1] + ... + arr[i])

Recurrence Relation

จากการวิเคราะห์ Optimal Substructure จะได้ว่า dp(n) = dp($n-1$) + arr[n] ในส่วน Base Case ซึ่งคือค่าเริ่มต้นที่ไม่ต้องคำนวณจาก state อื่น เรากำหนดว่า dp[0] = arr[0] นั่นคือในกรณีมีตัวเดียวผลรวมจากต้นถึงตัวมันเอง ก็คือตัวมันเอง

$$dp(n) = \begin{cases} 0 & \text{if } n = 0, \\ dp(n-1) + arr[n] & \text{if } n > 0. \end{cases}$$

เมื่อเรามี dp array แล้ว การตอบคำถาม sum(l, r) ทำได้ง่ายมาก: sum(l, r) = dp[r] - dp[$l-1$] (ถ้า $l = 0$, dp[$l-1$] จะกลายเป็น dp[-1] ซึ่งไม่มี ดังนั้นเราต้องแยกกรณีนี้: sum($0, r$) = dp[r])

Recursive Solution

วิธีนี้จะแปลง Recurrence Relation ไปเป็นฟังก์ชันทวนกลับโดยตรง แต่ไม่มีการจดจำผลลัพธ์

```
long long sumRecursive(int arr[], int i) {
    // Base Case
    if (i == 0) {
        return arr[0];
    }
    // Recursive Step
    return arr[i] + sumRecursive(arr, i - 1);
}
```

เวลาการทำงานเป็น $O(n)$ และเวลาในการสอบถามช่วงเป็น $O(n)$ เช่นกัน

Top-down Solution

วิธีนี้ปรับปรุงจากแบบ Recursive โดยการเพิ่ม "ความจำ" (memo) ให้กับฟังก์ชัน เพื่อไม่ให้คำนวณปัญหาย่อยเดิมซ้ำ

```
long long sumTopDown(int arr[], vector<long long>& memo, int i) {
    // Check memo
    if (memo[i] != -1) {
        return memo[i];
    }
    // Base Case
    if (i == 0) {
        memo[0] = arr[0];
        return memo[0];
    }
    // Compute and store the result in memo
    memo[i] = arr[i] + sumTopDown(arr, memo, i - 1);
    return memo[i];
}
```

เวลาการทำงานเป็น $O(n)$ แต่เวลาในการสอบถามช่วงเป็น $O(1)$

Bottom-up Solution

เราจะสร้าง dp array ขึ้นมาแล้วเติมค่าจากปัญหาย่อยที่เล็กที่สุดไปหาใหญ่ที่สุด ในบางครั้งเราอาจจะบวกสะสมพร้อมกับรับค่าไปด้วยก็ได้

```
#include <bits/stdc++.h>
using namespace std;
void buildPrefixSumBottomUp(int arr[], int n, vector<long long>& dp) {
    // Base Case
    dp[0] = arr[0];
    for (int i = 1; i < n; ++i) {
        dp[i] = dp[i - 1] + arr[i];
    }
}
```

```
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    vector<long long> dp(n);
    buildPrefixSumBottomUp(arr, n, dp);
    // Fast range sum query for interval [l, r]
    int l = 1, r = 3;
    // We want the sum of arr[1] + arr[2] + arr[3] = 2+3+4 = 9
    long long range_sum;
    if (l == 0) {
        range_sum = dp[r];
    } else {
        range_sum = dp[r] - dp[l - 1];
    }
    cout << range_sum << endl;
    return 0;
}
```

เวลาการทำงานเป็น $O(n)$ แต่เวลาในการสอบถามช่วงเป็น $O(1)$